Java programs

Submitted by :

Omama Tanveer

Reg No:

“2026-S-CYS-25”

Submitted to :

Sir Mubashir Sb

Task 1: Read an integer and print whether it is even or odd without using the % operator.

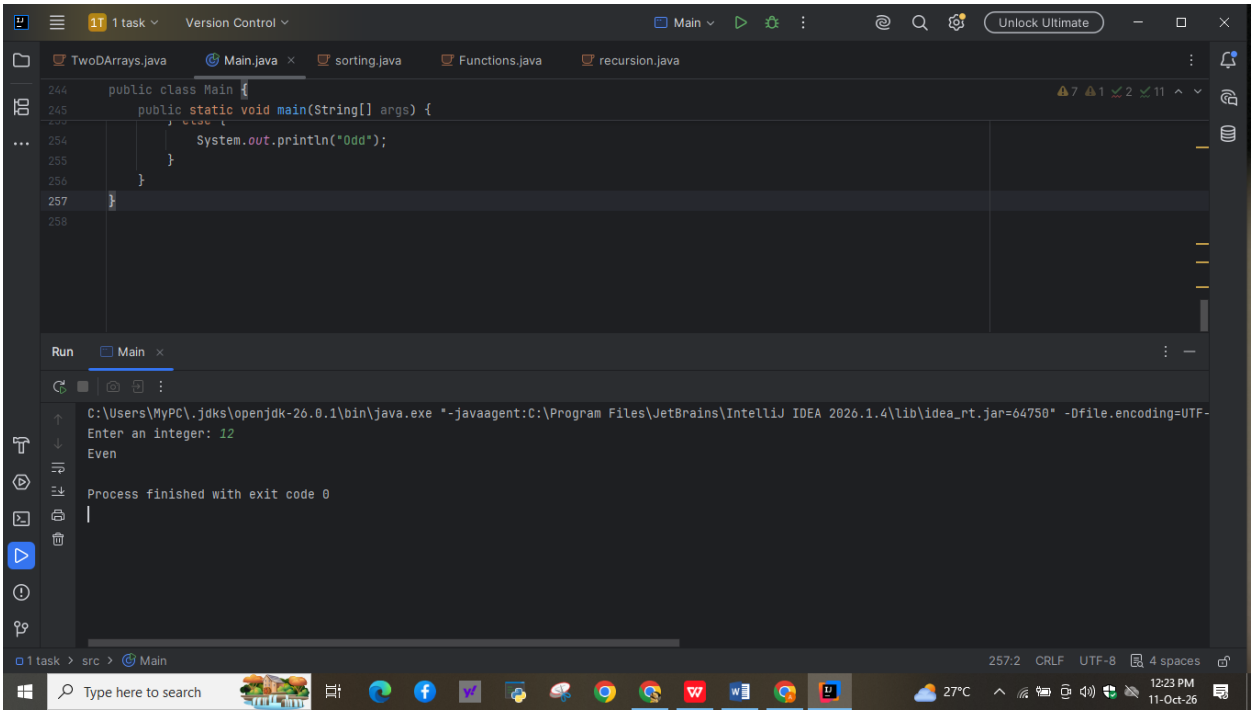
```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter an integer: ");
        int n = sc.nextInt();

        if ((n / 2) * 2 == n) {
            System.out.println("Even");
        } else {
            System.out.println("Odd");
        }
    }
}
```

Output:



Task 2: Swap two int variables without using a third variable. Print the values before and after swapping.

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter first number: ");
        int a = sc.nextInt();

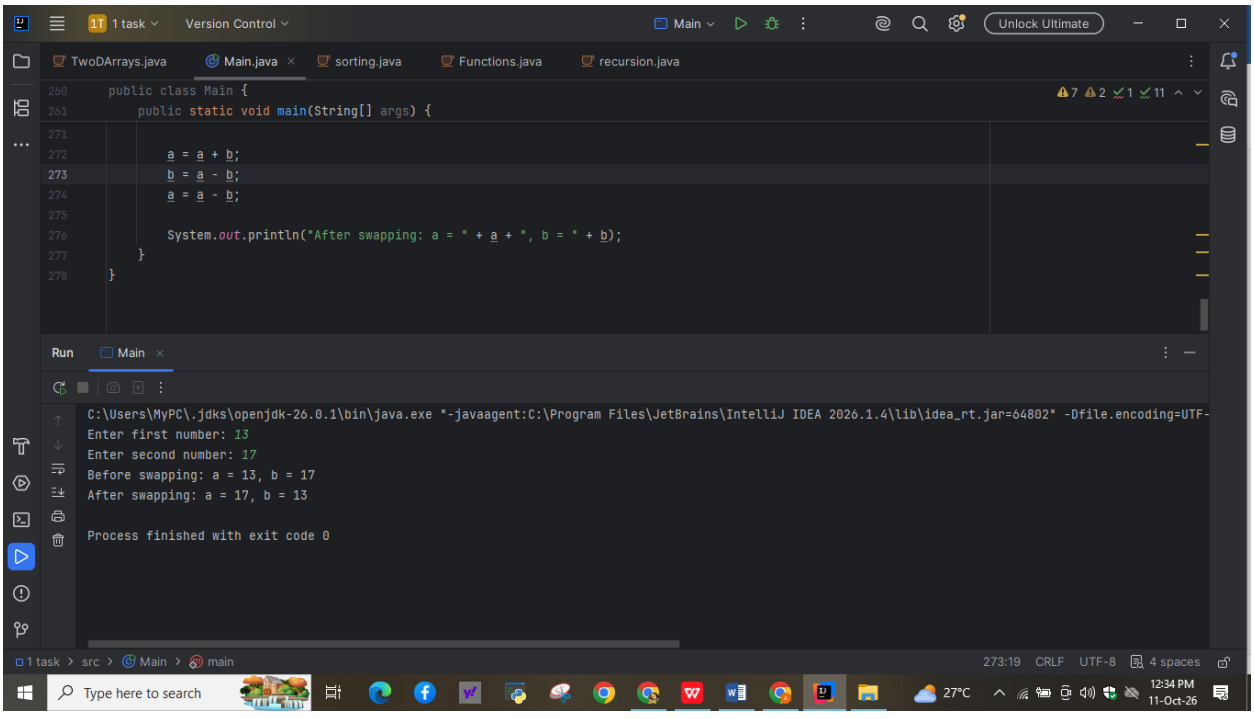
        System.out.print("Enter second number: ");
        int b = sc.nextInt();

        System.out.println("Before swapping: a = " + a + ", b = " + b);

        a = a + b;
        b = a - b;
        a = a - b;

        System.out.println("After swapping: a = " + a + ", b = " + b);
    }
}
```

Output:



Task 3: Read three numbers and print the middle (median) one without using arrays or sorting.

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

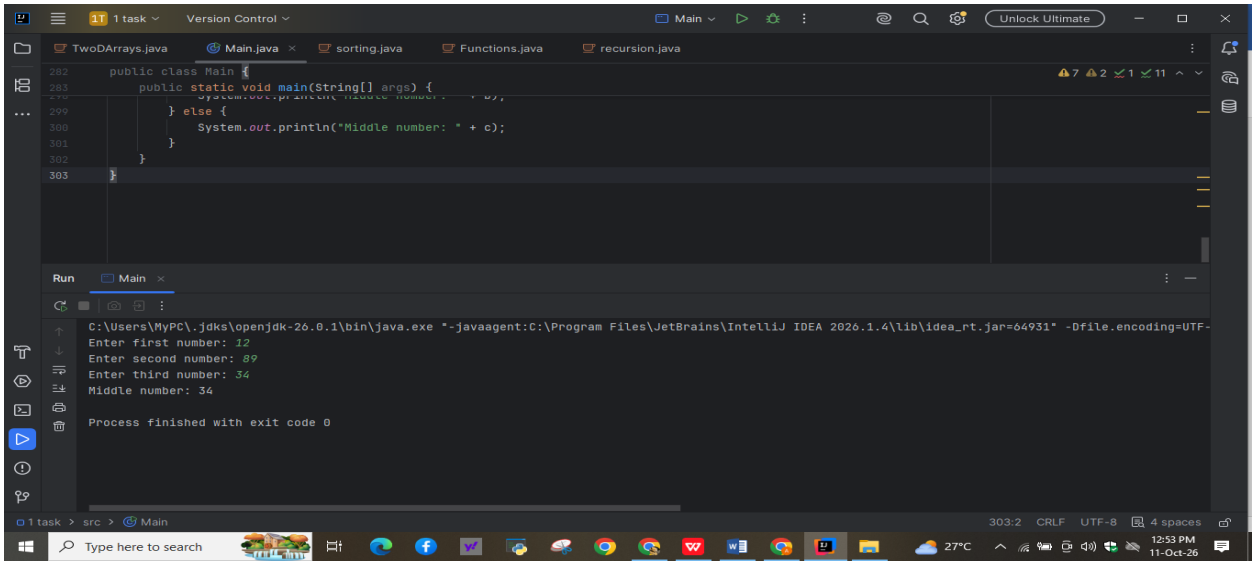
        System.out.print("Enter first number: ");
        int a = sc.nextInt();

        System.out.print("Enter second number: ");
        int b = sc.nextInt();

        System.out.print("Enter third number: ");
        int c = sc.nextInt();

        if ((a >= b && a <= c) || (a <= b && a >= c)) {
            System.out.println("Middle number: " + a);
        } else if ((b >= a && b <= c) || (b <= a && b >= c)) {
            System.out.println("Middle number: " + b);
        } else {
            System.out.println("Middle number: " + c);
        }
    }
}
```

Output:



Task 4:

Print a number triangle where row i contains the digit i repeated i times (1, 22, 333, ...) for a user-given n.

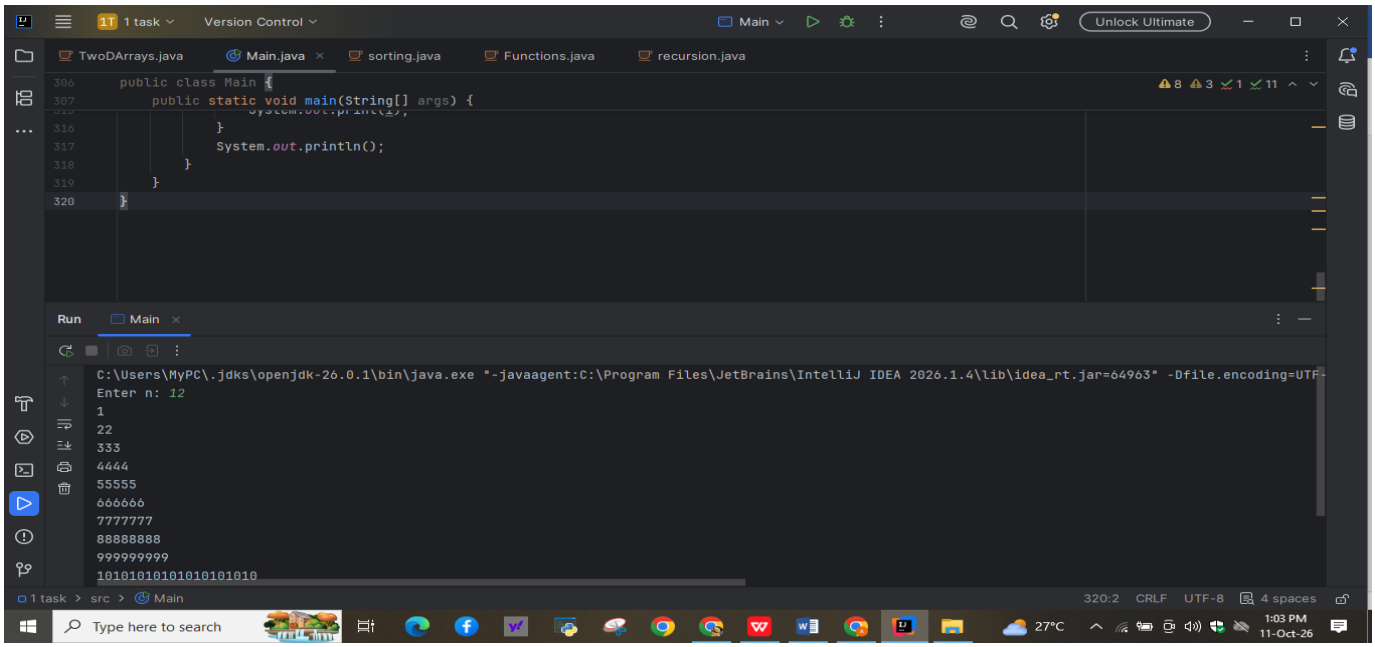
```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter n: ");
        int n = sc.nextInt();

        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= i; j++) {
                System.out.print(i);
            }
            System.out.println();
        }
    }
}
```

Output:



Task 5:

Read an integer and print its digits in reverse order without converting it to a String. Trailing zeros must be kept (1200 prints 0021).

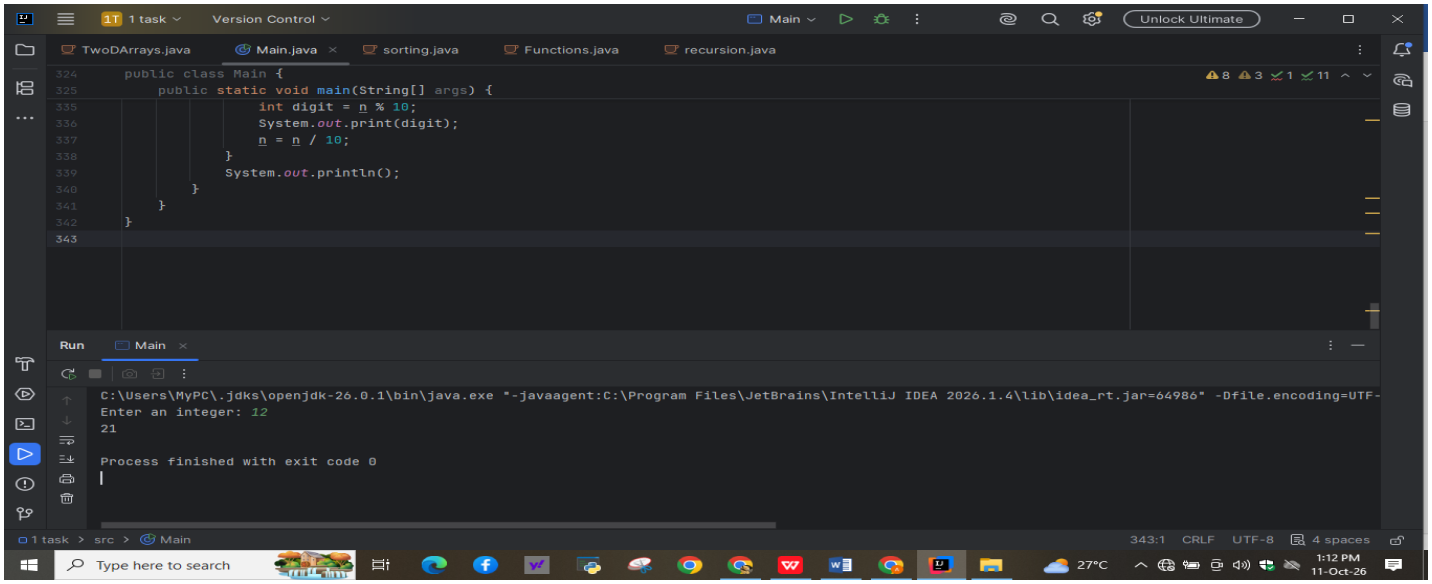
```
import java.util.Scanner;


public class Main {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);


      System.out.print("Enter an integer: ");
      int n = sc.nextInt();


      if (n == 0) {
         System.out.println(0);
      } else {
         while (n > 0) {
            int digit = n % 10;
            System.out.print(digit);
            n = n / 10;
         }
         System.out.println();
      }
   }
}
```

Output:



Task 6:

Repeatedly add the digits of a number until a single digit remains (for example 9875 gives 29, then 11, then 2).2

```
import java.util.Scanner;


public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

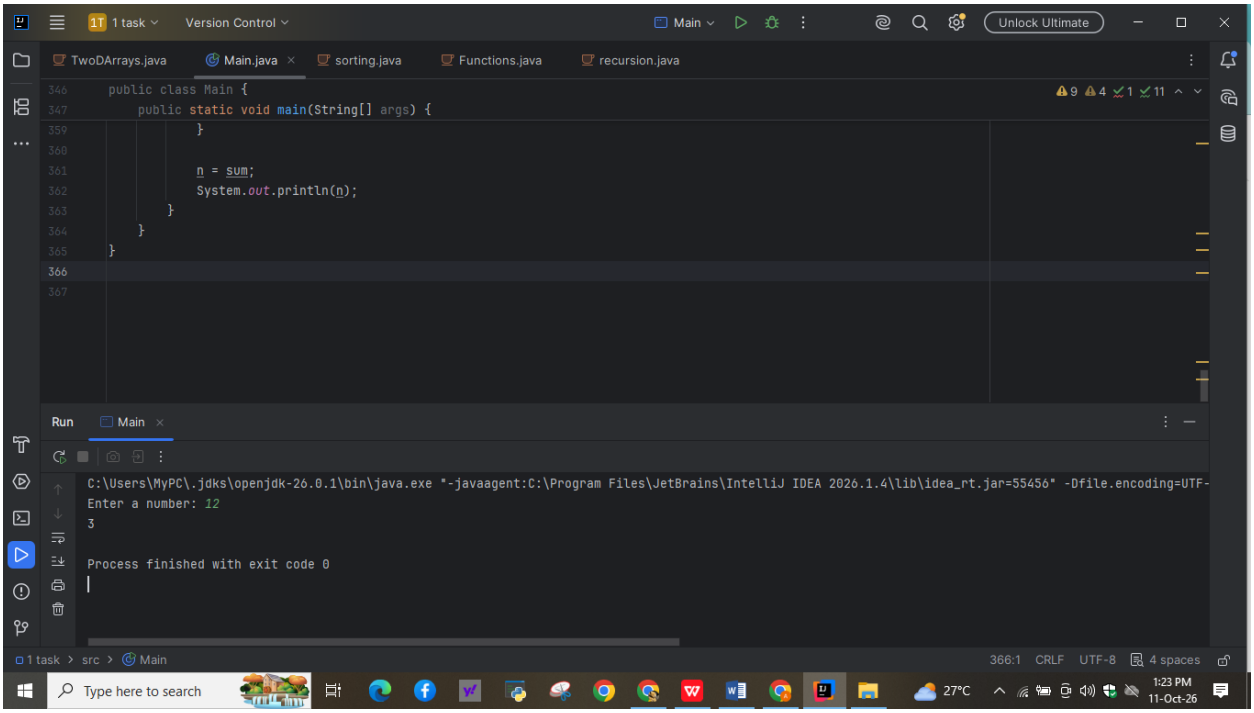

        System.out.print("Enter a number: ");
        int n = sc.nextInt();


        while (n >= 10) {
            int sum = 0;
```

```
        while (n > 0) {
            sum = sum + (n % 10);
            n = n / 10;
        }

        n = sum;
        System.out.println(n);
    }
  }
}
```

Output:



Task 7:

Write a method isPrime(int n) that loops only up to the square root of n. Use it to print all primes from 1 to 100.

```
public class Main {

    static boolean isPrime(int n) {
        if (n < 2) {
            return false;
        }

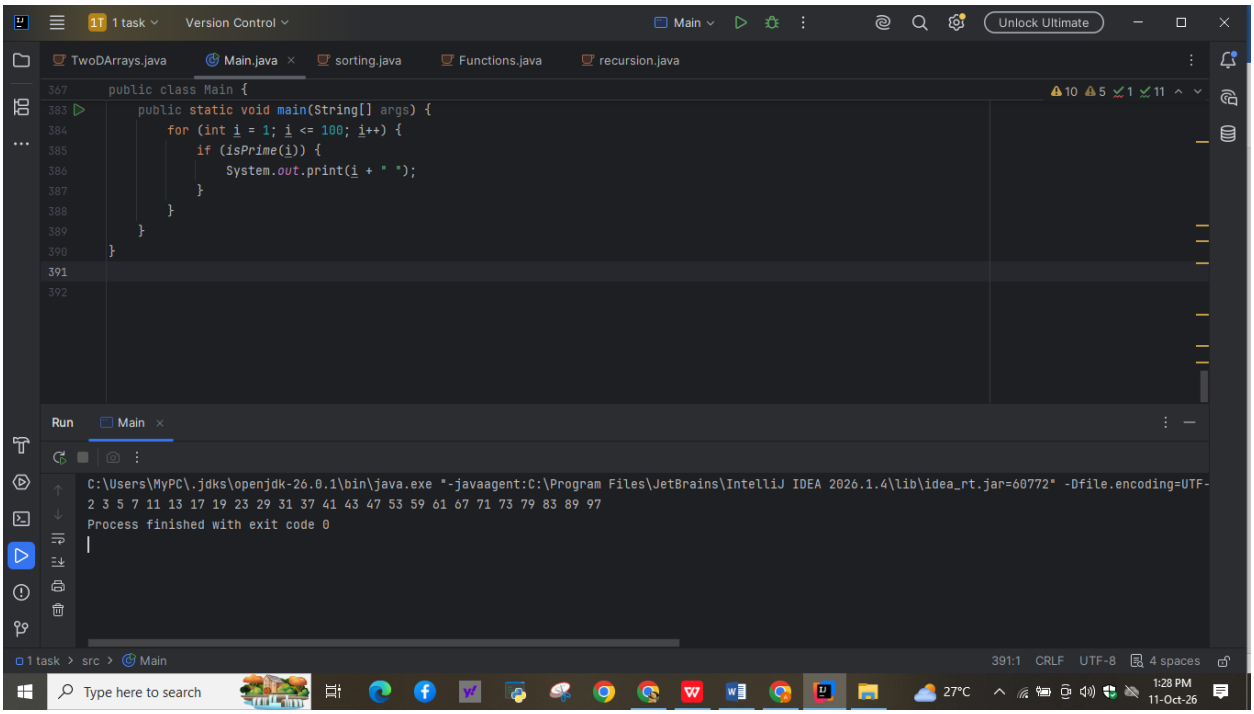
        for (int i = 2; i * i <= n; i++) {
            if (n % i == 0) {
                return false;
            }
        }

        return true;
    }

    public static void main(String[] args) {
        for (int i = 1; i <= 100; i++) {
            if (isPrime(i)) {
                System.out.print(i + " ");
            }
        }
    }
```

}

Output:



Task 8:

Read a String and count vowels, consonants, digits and other characters using a single loop.

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter a string: ");
        String str = sc.nextLine();

        int vowels = 0;
        int consonants = 0;
```

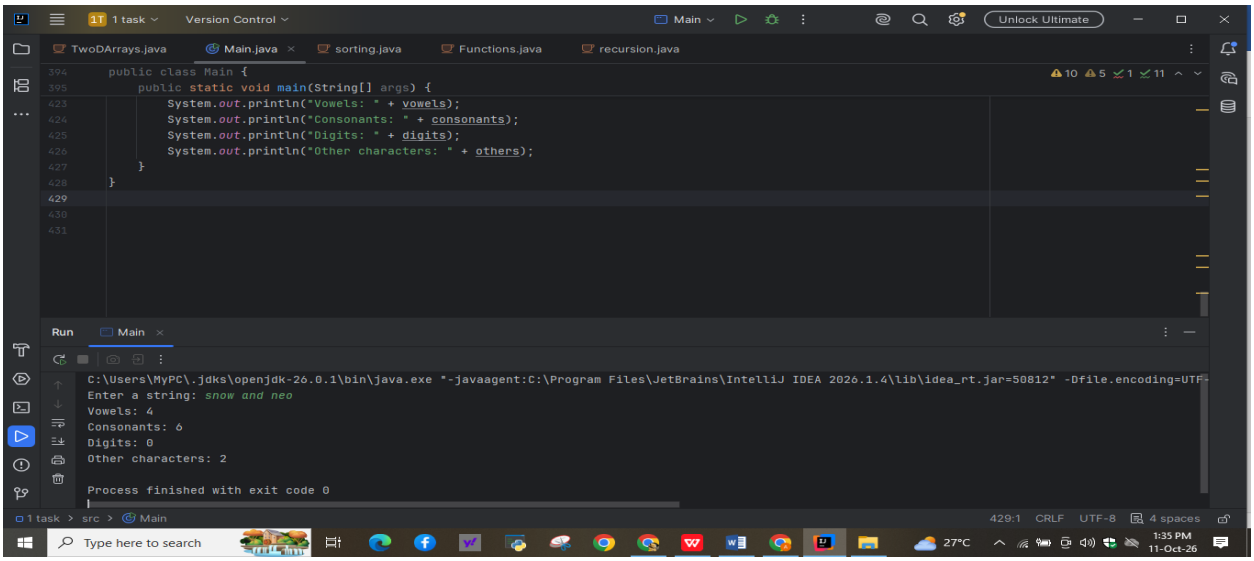
```
        int digits = 0;
        int others = 0;

        for (int i = 0; i < str.length(); i++) {
            char ch = Character.toLowerCase(str.charAt(i));

            if (ch >= 'a' && ch <= 'z') {
                if (ch == 'a' || ch == 'e' || ch == 'i'
                        || ch == 'o' || ch == 'u') {
                    vowels++;
                } else {
                    consonants++;
                }
            } else if (ch >= '0' && ch <= '9') {
                digits++;
            } else {
                others++;
            }
        }

        System.out.println("Vowels: " + vowels);
        System.out.println("Consonants: " + consonants);
        System.out.println("Digits: " + digits);
        System.out.println("Other characters: " + others);
    }
}
```

Output:



Task 9:

Print the first N Fibonacci numbers (N from the user) using only three variables, no arrays.

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter N: ");
        int n = sc.nextInt();

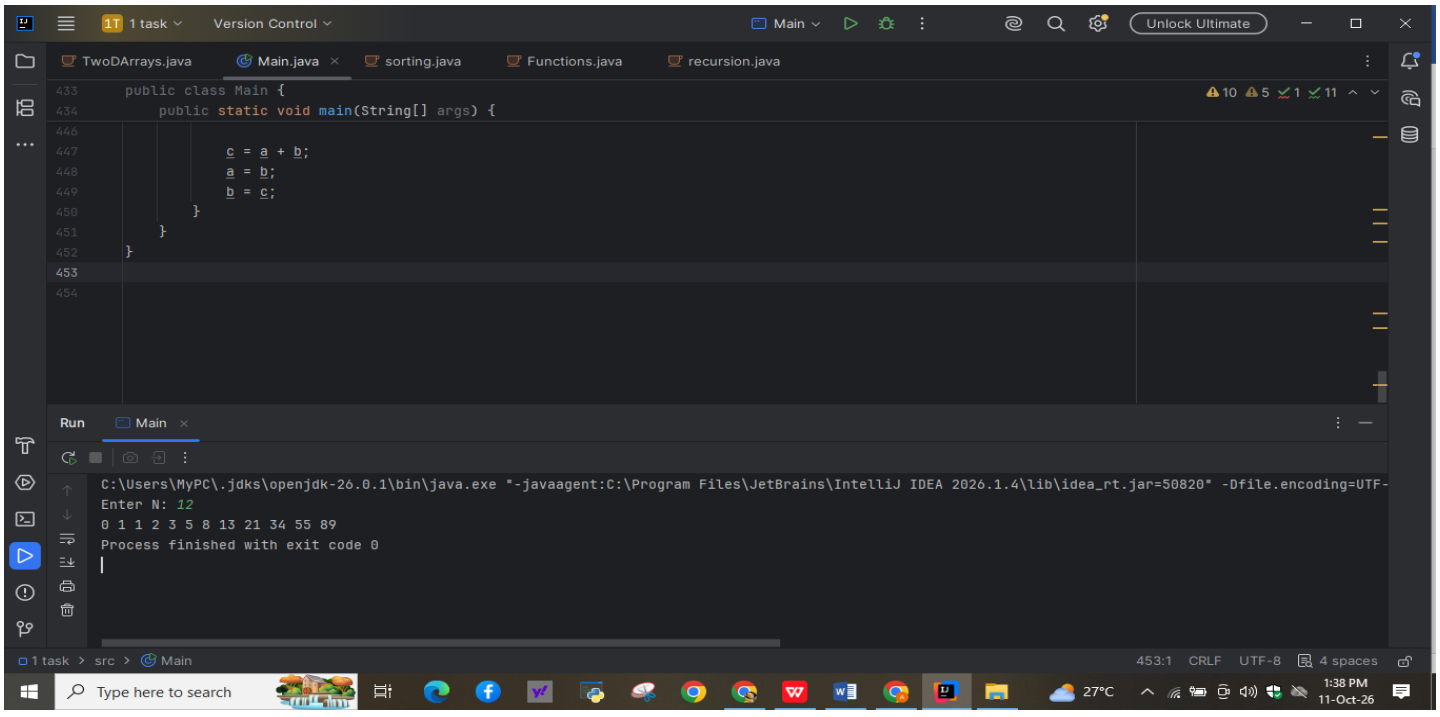
        int a = 0;
        int b = 1;
```

```
        int c;

        for (int i = 1; i <= n; i++) {
            System.out.print(a + " ");

            c = a + b;
            a = b;
            b = c;
        }
    }
}
```

Output:



Task 10:

Check whether a String is a palindrome, ignoring case, spaces and punctuation ("A man, a plan, a canal: Panama" must return true).

```
import java.util.Scanner;

public class Main {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);

      System.out.print("Enter a string: ");
      String str = sc.nextLine();

      str = str.toLowerCase();

      String clean = "";

      for (int i = 0; i < str.length(); i++) {
         char ch = str.charAt(i);

         if (ch >= 'a' && ch <= 'z' || ch >= '0' && ch <= '9') {
            clean = clean + ch;
         }
      }

       boolean palindrome = true;

      int left = 0;
      int right = clean.length() - 1;

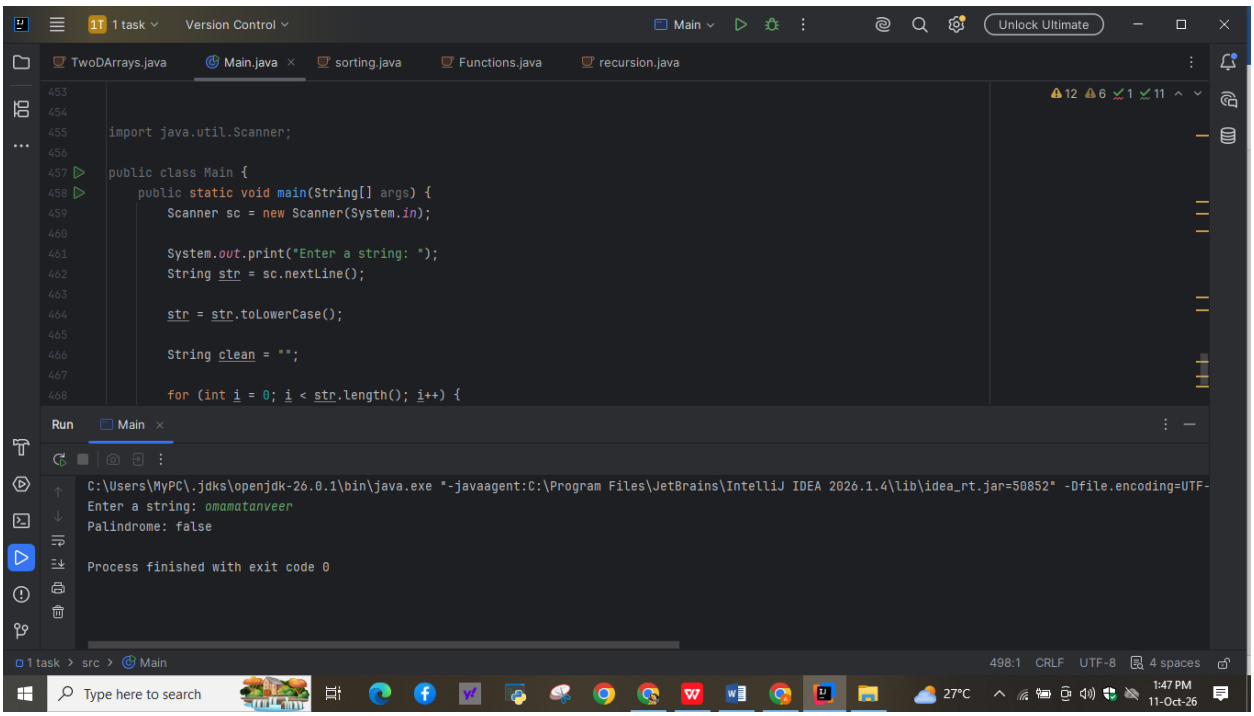
      while (left < right) {
         if (clean.charAt(left) != clean.charAt(right)) {
```

```
            palindrome = false;
            break;
        }

        left++;
        right--;
    }

    if (palindrome) {
        System.out.println("Palindrome: true");
    } else {
        System.out.println("Palindrome: false");
    }
  }
}
```

Output:



Task 11:

Find the second largest value in an int array of 10 elements without sorting. The array may contain duplicates of the largest value.

```
import java.util.Scanner;


public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);


        int[] arr = new int[10];


        System.out.println("Enter 10 integers:");

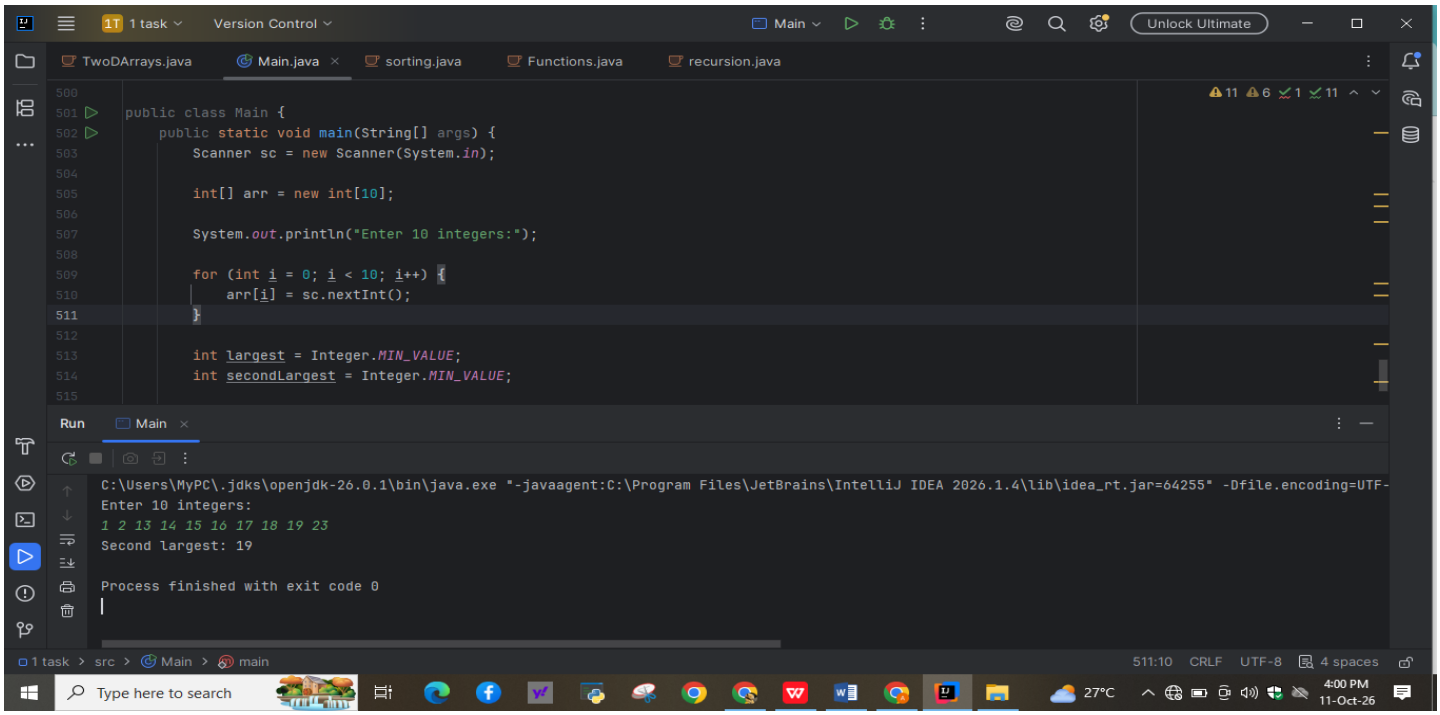

        for (int i = 0; i < 10; i++) {
            arr[i] = sc.nextInt();
        }


        int largest = Integer.MIN_VALUE;
        int secondLargest = Integer.MIN_VALUE;


        for (int i = 0; i < 10; i++) {
            if (arr[i] > largest) {
                secondLargest = largest;
                largest = arr[i];
            } else if (arr[i] < largest && arr[i] > secondLargest) {
                secondLargest = arr[i];
            }
        }
```

```
        if (secondLargest == Integer.MIN_VALUE) {
            System.out.println("No second largest distinct value exists.");
        } else {
            System.out.println("Second largest: " + secondLargest);
        }
    }
}
```

Output:



Task 12 :

Rotate an array to the left by k positions in place, without creating a second array (k may be larger than the array length).

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter array size: ");
        int n = sc.nextInt();

        int[] arr = new int[n];

        System.out.println("Enter array elements:");
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }

        System.out.print("Enter k: ");
        int k = sc.nextInt();

        if (n > 0) {
            k = k % n;

            // Reverse the first k elements
            reverse(arr, 0, k - 1);

            // Reverse the remaining elements
            reverse(arr, k, n - 1);

            // Reverse the entire array
```

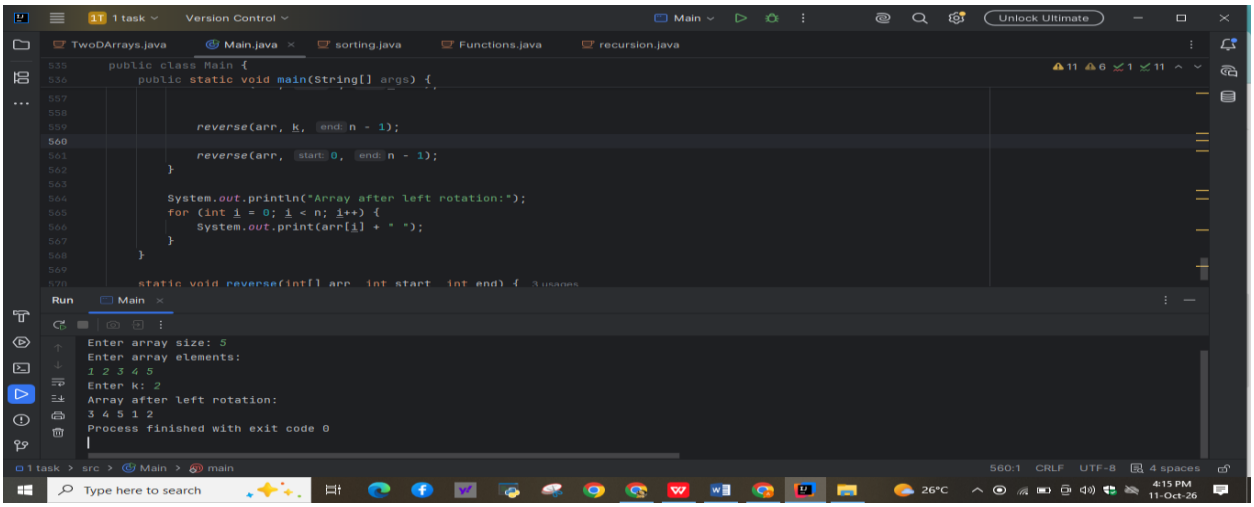
```
        reverse(arr, 0, n - 1);
    }

    System.out.println("Array after left rotation:");
    for (int i = 0; i < n; i++) {
        System.out.print(arr[i] + " ");
    }
  }

  static void reverse(int[] arr, int start, int end) {
    while (start < end) {
        int temp = arr[start];
        arr[start] = arr[end];
        arr[end] = temp;

        start++;
        end--;
    }
  }
}
```

Output:



Task 13:

Print a hollow diamond of stars for an odd n given by the user.

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter an odd number: ");
        int n = sc.nextInt();

        for (int i = 1; i <= n; i += 2) {
            for (int s = 1; s <= (n - i) / 2; s++) {
                System.out.print(" ");
            }

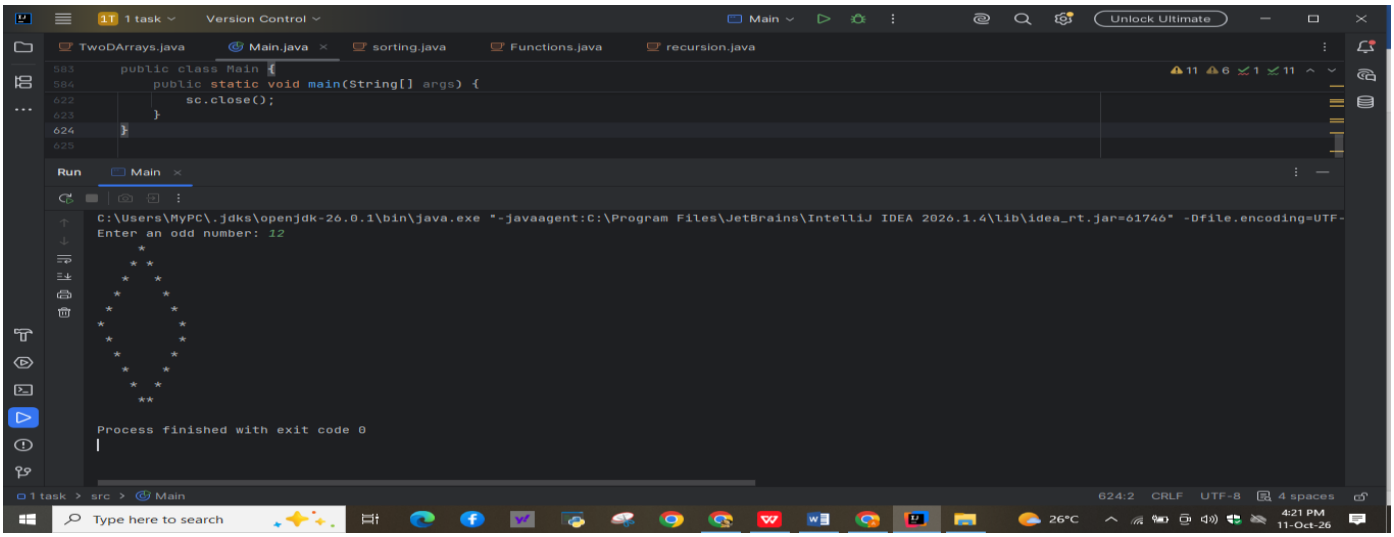
            for (int j = 1; j <= i; j++) {
                if (j == 1 || j == i) {
                    System.out.print("*");
                } else if (i == n) {
                    System.out.print("*");
                } else {
                    System.out.print(" ");
                }
            }
            System.out.println();
        }

        for (int i = n - 2; i >= 1; i -= 2) {
            for (int s = 1; s <= (n - i) / 2; s++) {
                System.out.print(" ");
            }

            for (int j = 1; j <= i; j++) {
                if (j == 1 || j == i) {
                    System.out.print("*");
                } else {
                    System.out.print(" ");
                }
            }
            System.out.println();
        }

        sc.close();
    }
}
```

Output:



Task 14: Check whether two Strings are anagrams without using any sorting method.

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter first string: ");
        String s1 = sc.nextLine().toLowerCase();

        System.out.print("Enter second string: ");
        String s2 = sc.nextLine().toLowerCase();

        if (s1.length() != s2.length()) {
            System.out.println("Not Anagrams");
        } else {
            int[] count = new int[26];

            for (int i = 0; i < s1.length(); i++) {
                count[s1.charAt(i) - 'a']++;
                count[s2.charAt(i) - 'a']--;
            }

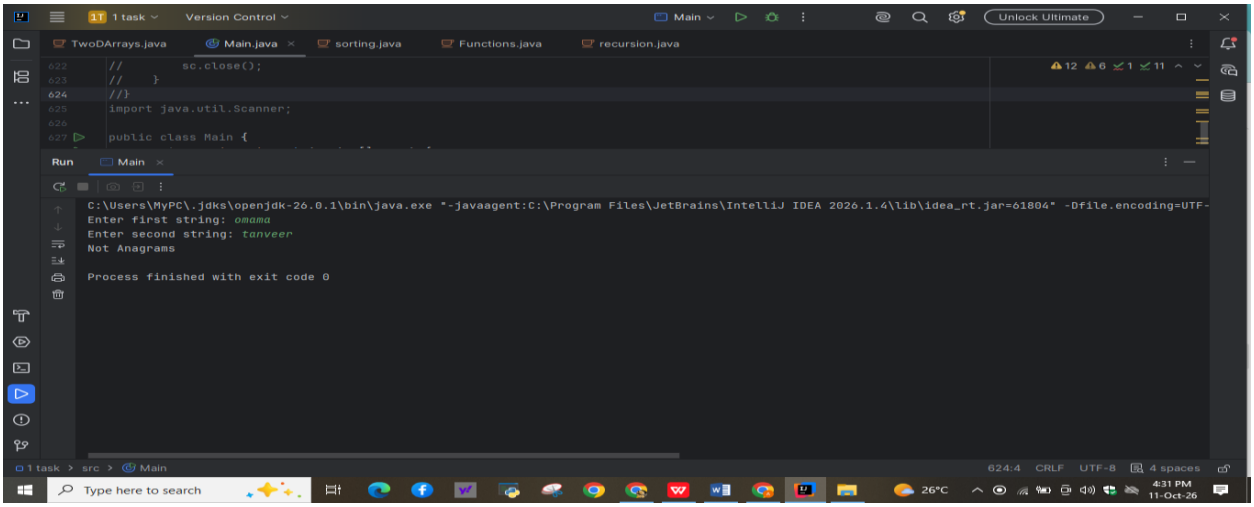
            boolean anagram = true;

            for (int i = 0; i < 26; i++) {
                if (count[i] != 0) {
                    anagram = false;
                    break;
                }
            }
```

```
            if (anagram) {
                System.out.println("Anagrams");
            } else {
                System.out.println("Not Anagrams");
            }
        }

        sc.close();
    }
}
```

Output:



Task 15:

Remove duplicates from an int array while preserving the original order. Print the result without using any Collection class.

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter array size: ");
        int n = sc.nextInt();

        int[] arr = new int[n];

        System.out.println("Enter array elements:");
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }

        System.out.print("Array after removing duplicates: ");

        for (int i = 0; i < n; i++) {
            boolean duplicate = false;

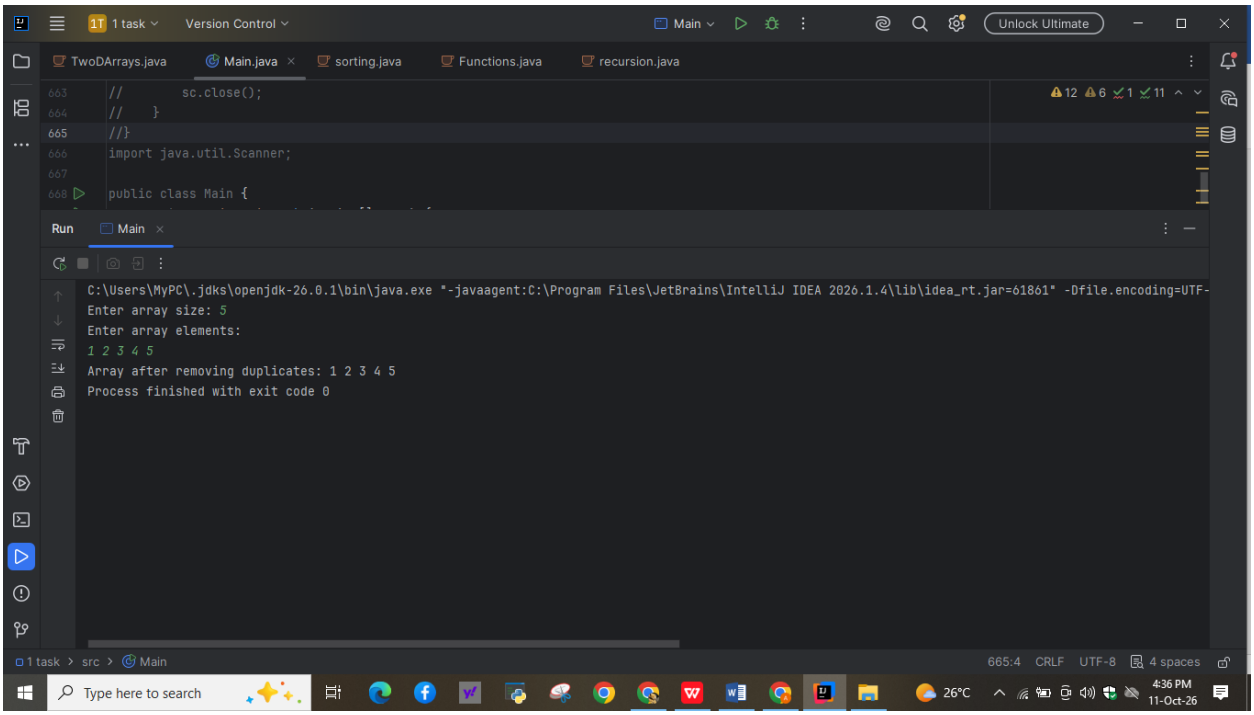
            for (int j = 0; j < i; j++) {
```

```
                if (arr[i] == arr[j]) {
                    duplicate = true;
                    break;
                }
            }

            if (!duplicate) {
                System.out.print(arr[i] + " ");
            }
        }

        sc.close();
    }
}
```

Output:



Task 16:

Write a recursive method that computes x^n using fast exponentiation. Also count and print how many multiplications were performed for n = 30.

```
public class Main {

    static int multiplications = 0;

    static long power(long x, int n) {
        if (n == 0) {
            return 1;
        }
```

```
        long half = power(x, n / 2);

        multiplications++;
        long result = half * half;

        if (n % 2 != 0) {
            multiplications++;
            result = result * x;
        }

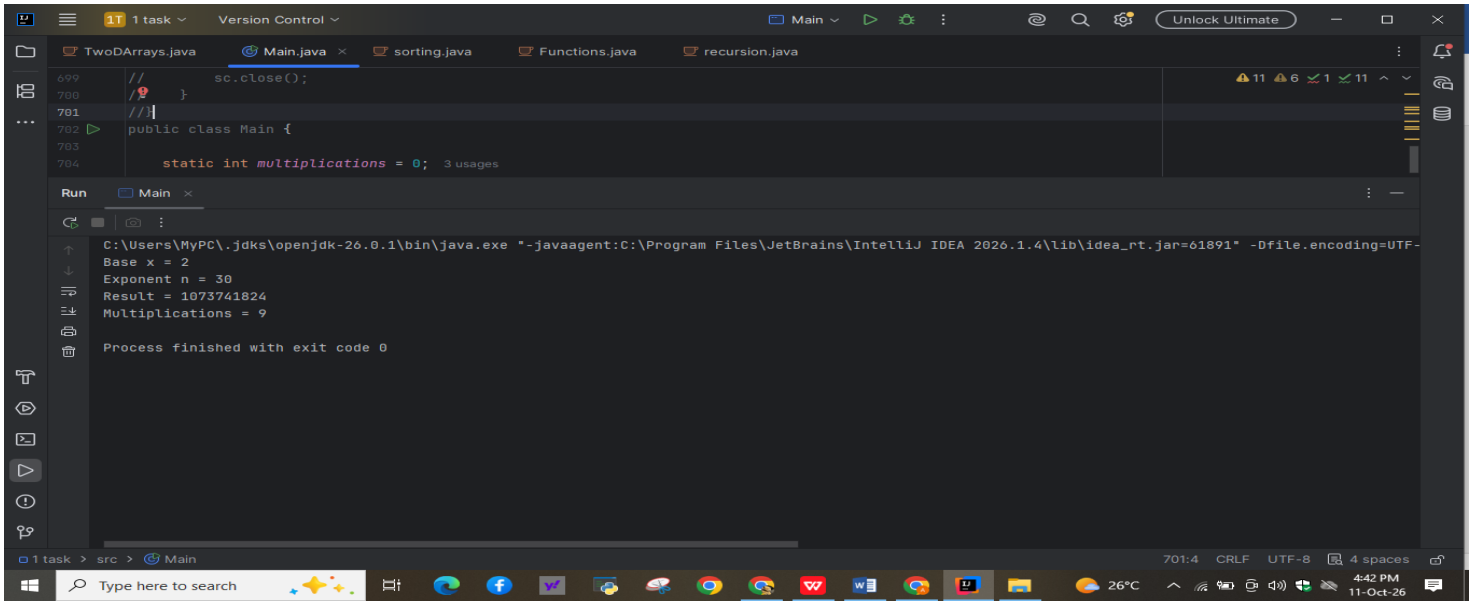
        return result;
    }

    public static void main(String[] args) {
        long x = 2;
        int n = 30;

        long result = power(x, n);

        System.out.println("Base x = " + x);
        System.out.println("Exponent n = " + n);
        System.out.println("Result = " + result);
        System.out.println("Multiplications = " + multiplications);
    }
}
```

Output:



Task 17: Read a 3x3 matrix, print its transpose, and report whether the matrix is symmetric.

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int[][] matrix = new int[3][3];
        boolean symmetric = true;

        System.out.println("Enter 9 matrix elements:");

        // Input matrix
        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 3; j++) {
                matrix[i][j] = sc.nextInt();
            }
        }

        // Print transpose
        System.out.println("Transpose of the matrix:");

        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 3; j++) {
                System.out.print(matrix[j][i] + " ");
            }
            System.out.println();
        }

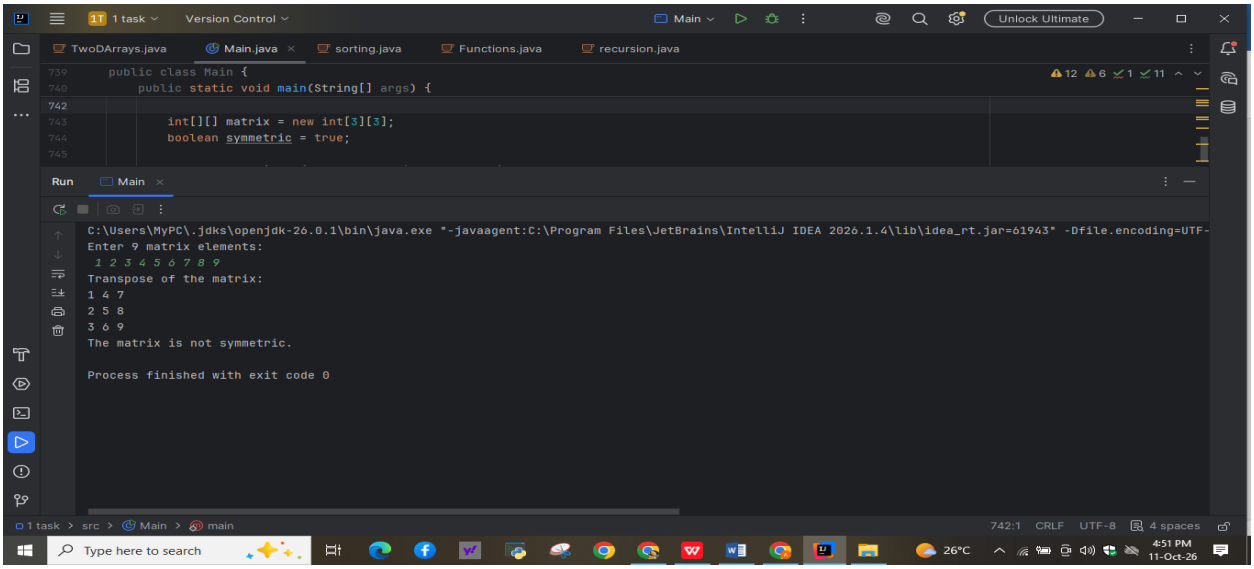
        // Check symmetry
        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 3; j++) {
                if (matrix[i][j] != matrix[j][i]) {
                    symmetric = false;
                    break;
                }
            }
        }

        // Print result
        if (symmetric) {
            System.out.println("The matrix is symmetric.");
        } else {
            System.out.println("The matrix is not symmetric.");
        }

        sc.close();
```

```
    }
}
```

Output:



Task 18:

Convert a decimal number to binary without using Integer.toBinaryString(). Solve it once with a loop and once with recursion.

```
import java.util.Scanner;

public class Main {

    // Method 1: Using a loop
    static String binaryLoop(int n) {
        if (n == 0) {
            return "0";
        }

        String binary = "";

        while (n > 0) {
            int remainder = n % 2;
            binary = remainder + binary;
            n = n / 2;
        }

        return binary;
    }
```

```
    // Method 2: Using recursion
    static String binaryRecursion(int n) {
        if (n < 2) {
            return String.valueOf(n);
        }

        return binaryRecursion(n / 2) + (n % 2);
    }

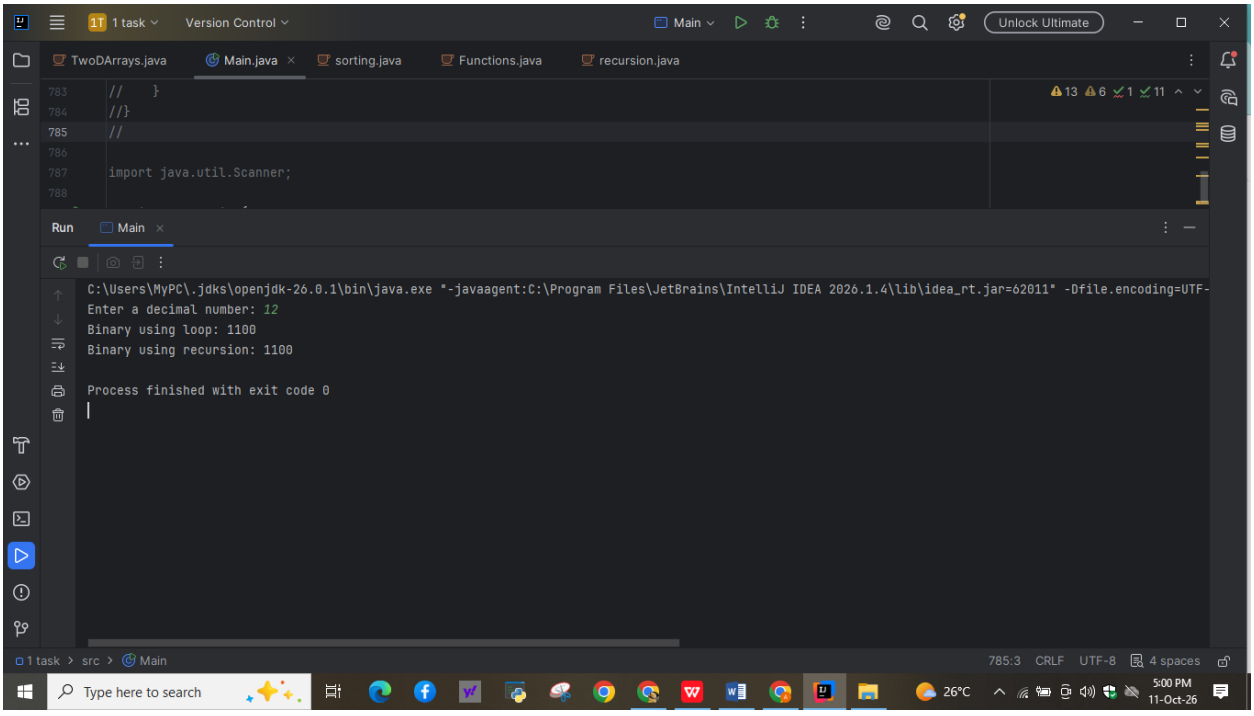
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter a decimal number: ");
        int n = sc.nextInt();

        if (n < 0) {
            System.out.println("Please enter a non-negative number.");
        } else {
            System.out.println("Binary using loop: " + binaryLoop(n));
            System.out.println("Binary using recursion: " + binaryRecursion(n));
        }

        sc.close();
    }
}
```

Output:



Task 19:

Print the first non-repeating character of a String, or "none" if every character repeats.

```
import java.util.Scanner;

public class Main {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);

      System.out.print("Enter a string: ");
      String str = sc.nextLine();

      boolean found = false;

      for (int i = 0; i < str.length(); i++) {
         char ch = str.charAt(i);
         int count = 0;

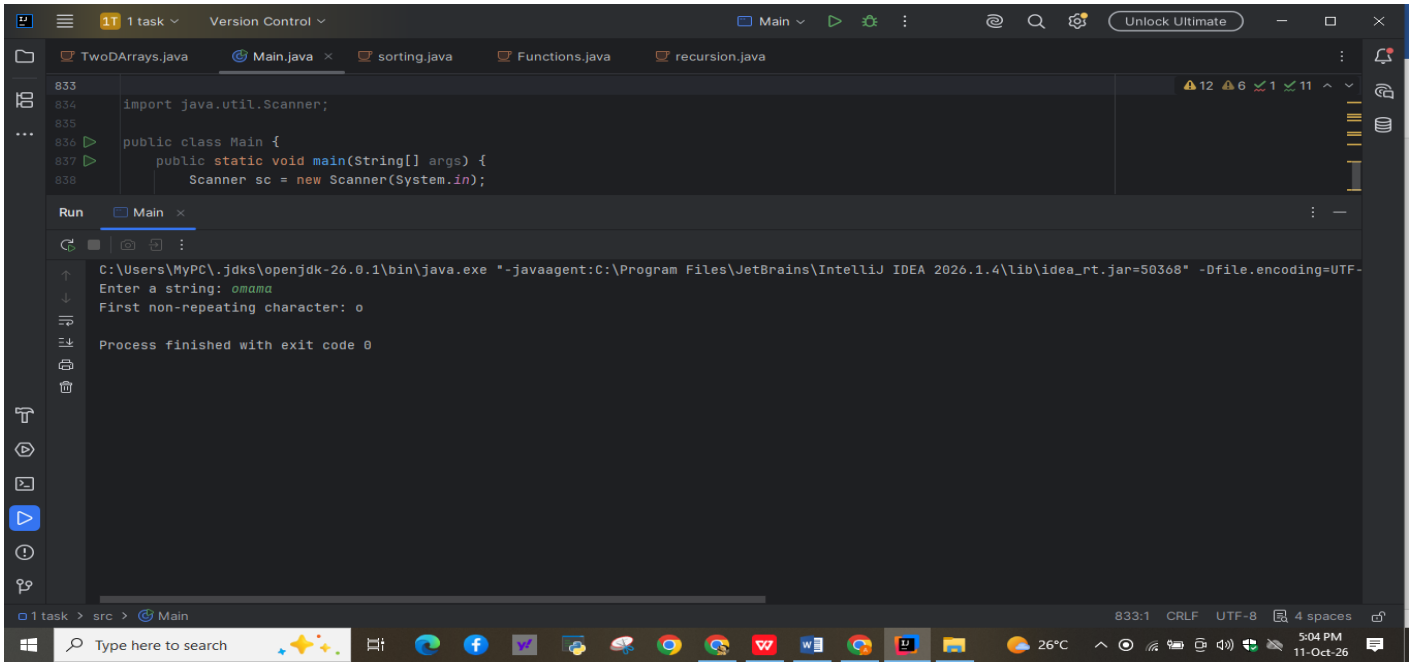
         for (int j = 0; j < str.length(); j++) {
            if (str.charAt(j) == ch) {
               count++;
            }
         }

         if (count == 1) {
            System.out.println("First non-repeating character: " + ch);
            found = true;
            break;
         }
      }

      if (!found) {
         System.out.println("none");
      }

      sc.close();
   }
}
```

Output:



Task 20 : Print all Armstrong numbers between 1 and 1000.

```
public class Main {
    public static void main(String[] args) {

        System.out.println("Armstrong numbers between 1 and 1000:");

        for (int i = 1; i <= 1000; i++) {
            int num = i;
            int temp = num;
            int sum = 0;
            int digits = 0;

            // Count digits
            while (temp > 0) {
                digits++;
                temp = temp / 10;
            }

            temp = num;

            // Calculate the sum of powered digits
```

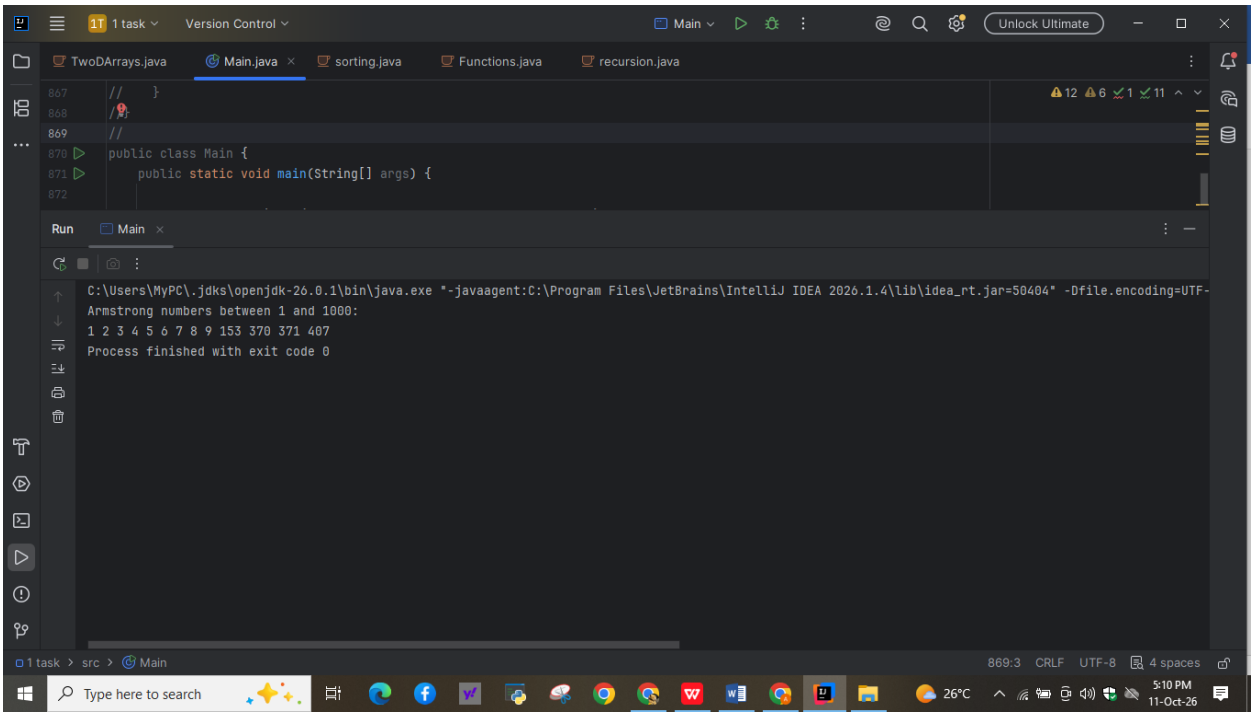
```
            while (temp > 0) {
                int digit = temp % 10;
                int power = 1;

                for (int j = 1; j <= digits; j++) {
                    power = power * digit;
                }

                sum = sum + power;
                temp = temp / 10;
            }

            // Check Armstrong number
            if (sum == num) {
                System.out.print(num + " ");
            }
        }
    }
}
```

Output:



Task 21:

Print Pascal's triangle for n rows using a jagged 2D array.

```
import java.util.Scanner;

public class Main {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);

      System.out.print("Enter number of rows: ");
      int n = sc.nextInt();

      int[][] triangle = new int[n][];

      // Create and fill the jagged array
      for (int i = 0; i < n; i++) {
         triangle[i] = new int[i + 1];

         for (int j = 0; j <= i; j++) {
            if (j == 0 || j == i) {
               triangle[i][j] = 1;
            } else {
               triangle[i][j] =
                     triangle[i - 1][j - 1]
                     + triangle[i - 1][j];
            }
         }
      }

      // Print Pascal's Triangle
      for (int i = 0; i < n; i++) {
         for (int s = 0; s < n - i - 1; s++) {
            System.out.print(" ");
         }

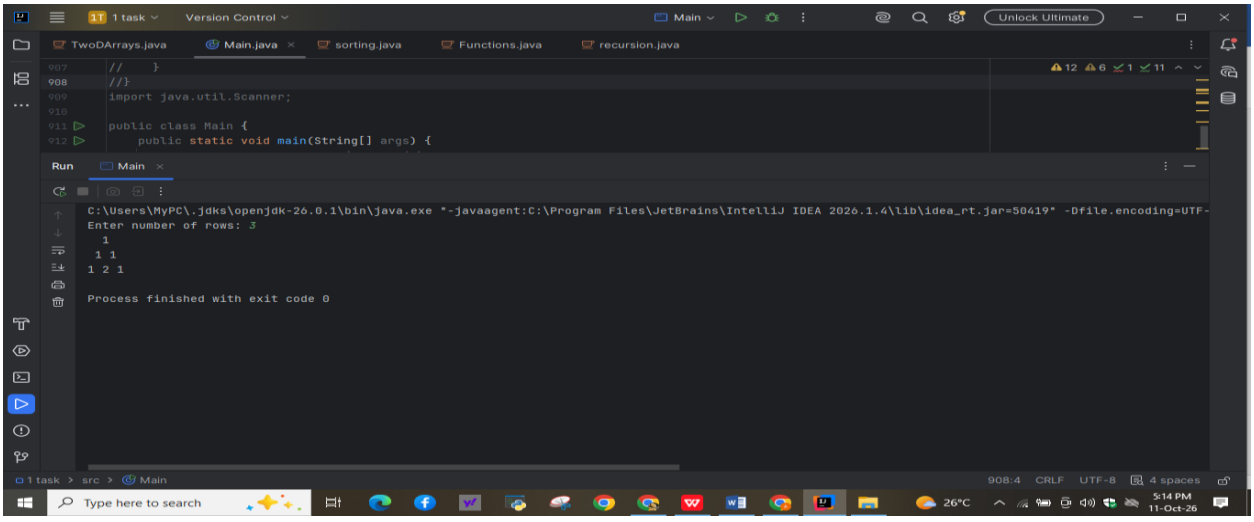
         for (int j = 0; j < triangle[i].length; j++) {
            System.out.print(triangle[i][j] + " ");
         }

         System.out.println();
      }

      sc.close();
   }


}
```

Output:



Task 22:

Generate and print all permutations of a String with distinct characters using recursion.

```
import java.util.Scanner;

public class Main {

    static void permute(String str, String result) {

        if (str.length() == 0) {
            System.out.println(result);
            return;
        }

        for (int i = 0; i < str.length(); i++) {

            char ch = str.charAt(i);

            String remaining = str.substring(0, i)
                    + str.substring(i + 1);

            permute(remaining, result + ch);
        }
    }

    public static void main(String[] args) {
```

```
        Scanner sc = new Scanner(System.in);

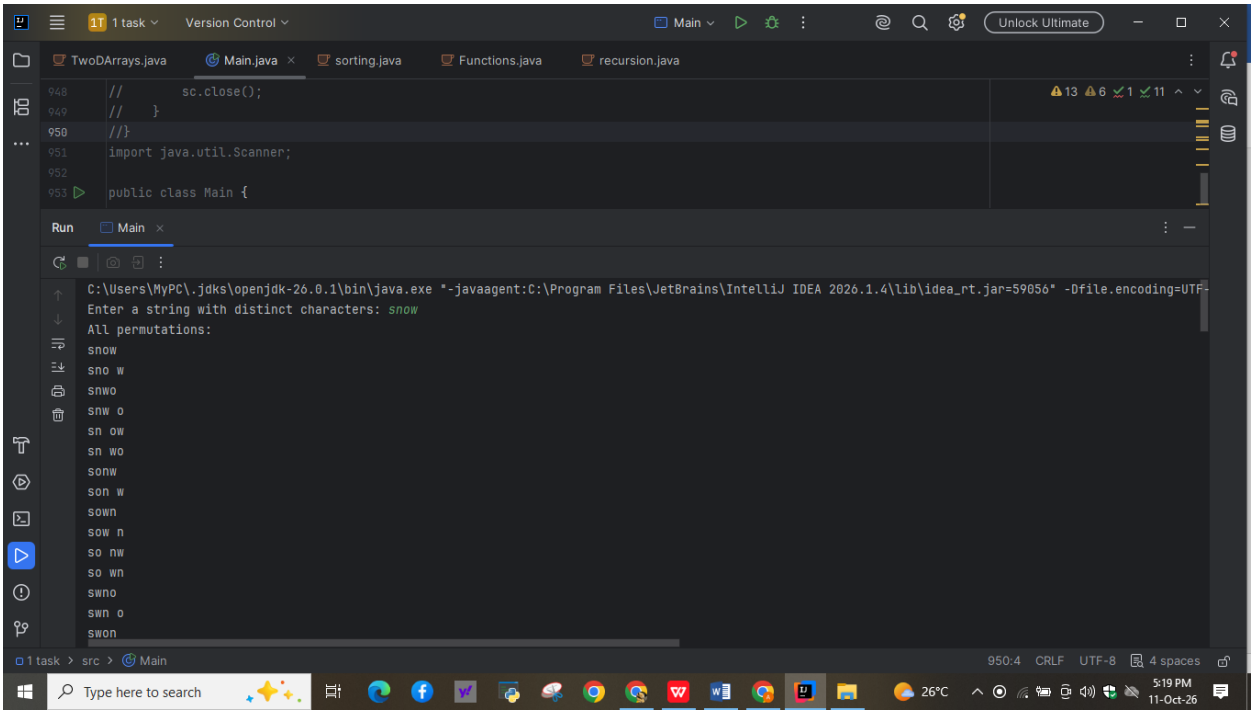
        System.out.print("Enter a string with distinct characters: ");
        String str = sc.nextLine();

        System.out.println("All permutations:");

        permute(str, "");

        sc.close();
    }
}
```

Output:



Task 23:

Write an iterative binary search that also returns the number of steps taken. Then modify it to return the first occurrence in an array containing duplicates.

```
import java.util.Scanner;
```

```
public class Main {

    // Task 1: Binary search with step count
    static int[] binarySearch(int[] arr, int target) {
        int low = 0;
        int high = arr.length - 1;
        int steps = 0;

        while (low <= high) {
            steps++;

            int mid = low + (high - low) / 2;

            if (arr[mid] == target) {
                return new int[]{mid, steps};
            } else if (arr[mid] < target) {
                low = mid + 1;
            } else {
                high = mid - 1;
            }
        }

        return new int[]{-1, steps};
    }

    // Task 2: Find first occurrence of target
    static int[] firstOccurrence(int[] arr, int target) {
        int low = 0;
        int high = arr.length - 1;
        int steps = 0;
        int first = -1;

        while (low <= high) {
            steps++;

            int mid = low + (high - low) / 2;

            if (arr[mid] == target) {
                first = mid;
                high = mid - 1;
            } else if (arr[mid] < target) {
                low = mid + 1;
            } else {
                high = mid - 1;
            }
        }

        return new int[]{first, steps};
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
```

```
        int[] arr = {1, 2, 2, 2, 4, 5, 7};

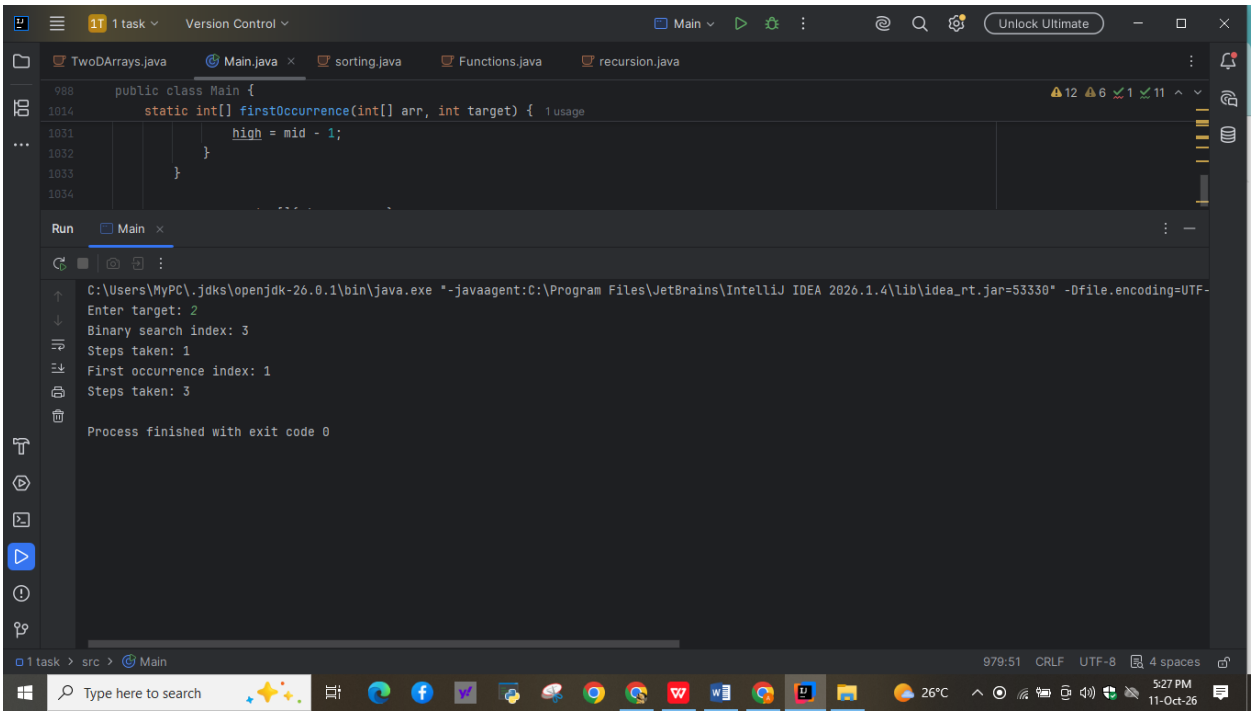
        System.out.print("Enter target: ");
        int target = sc.nextInt();

        int[] result1 = binarySearch(arr, target);
        System.out.println("Binary search index: " + result1[0]);
        System.out.println("Steps taken: " + result1[1]);

        int[] result2 = firstOccurrence(arr, target);
        System.out.println("First occurrence index: " + result2[0]);
        System.out.println("Steps taken: " + result2[1]);

        sc.close();
    }
}
```

Output:



Task 24:

Find the longest substring without repeating characters in a given String. Print the substring and its length.

```
import java.util.Scanner;

public class Main {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);

      System.out.print("Enter a string: ");
      String str = sc.nextLine();

      int maxLength = 0;
      int start = 0;
      int bestStart = 0;

      for (int i = 0; i < str.length(); i++) {

         for (int j = start; j < i; j++) {
            if (str.charAt(j) == str.charAt(i)) {
               start = j + 1;
               break;
            }
         }

         int length = i - start + 1;

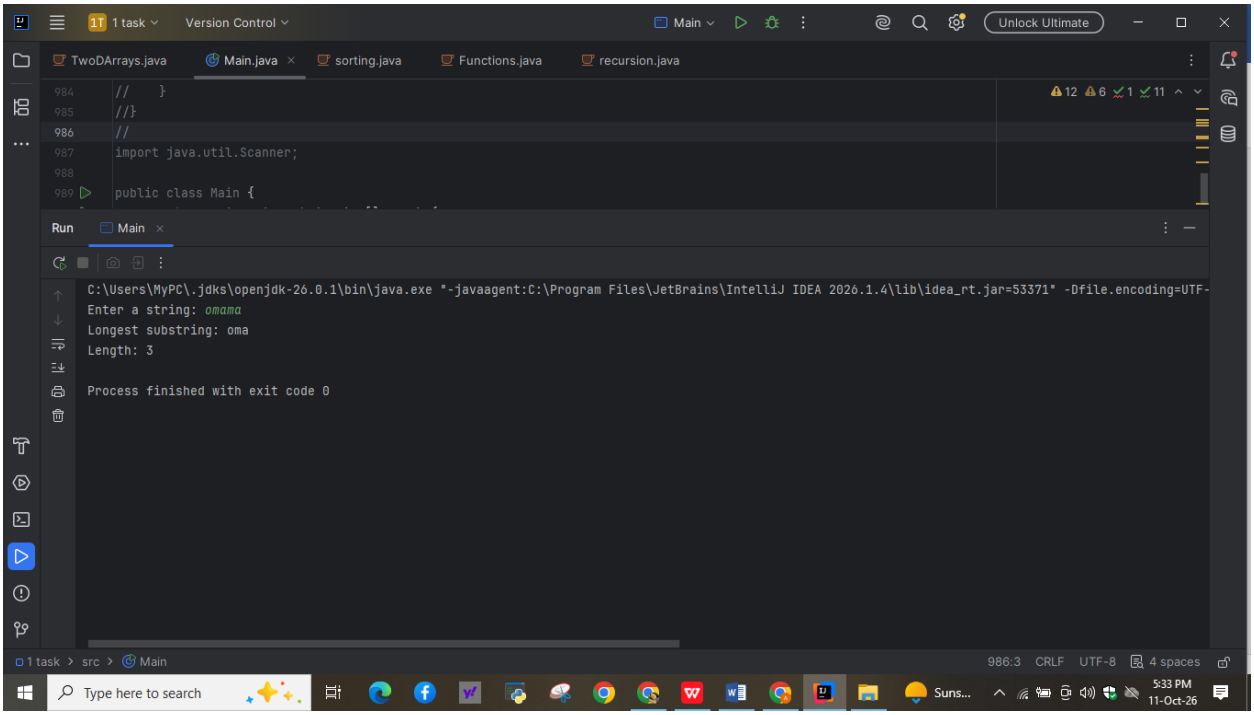
         if (length > maxLength) {
            maxLength = length;
            bestStart = start;
         }
      }

      String longest = str.substring(bestStart, bestStart + maxLength);

      System.out.println("Longest substring: " + longest);
      System.out.println("Length: " + maxLength);

      sc.close();
   }
}
```

Output:



Task 25:

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter rows: ");
        int n = sc.nextInt();

        System.out.print("Enter columns: ");
        int m = sc.nextInt();

        int[][] arr = new int[n][m];

        System.out.println("Enter matrix elements:");

        for (int i = 0; i < n; i++) {
            for (int j = 0; j < m; j++) {
                arr[i][j] = sc.nextInt();
            }
        }

        int top = 0;
        int bottom = n - 1;
        int left = 0;
        int right = m - 1;

        System.out.println("Spiral order:");
```

```
        while (top <= bottom && left <= right) {

            // 1. Print top row (left to right)
            for (int j = left; j <= right; j++) {
                System.out.print(arr[top][j] + " ");
            }
            top++;

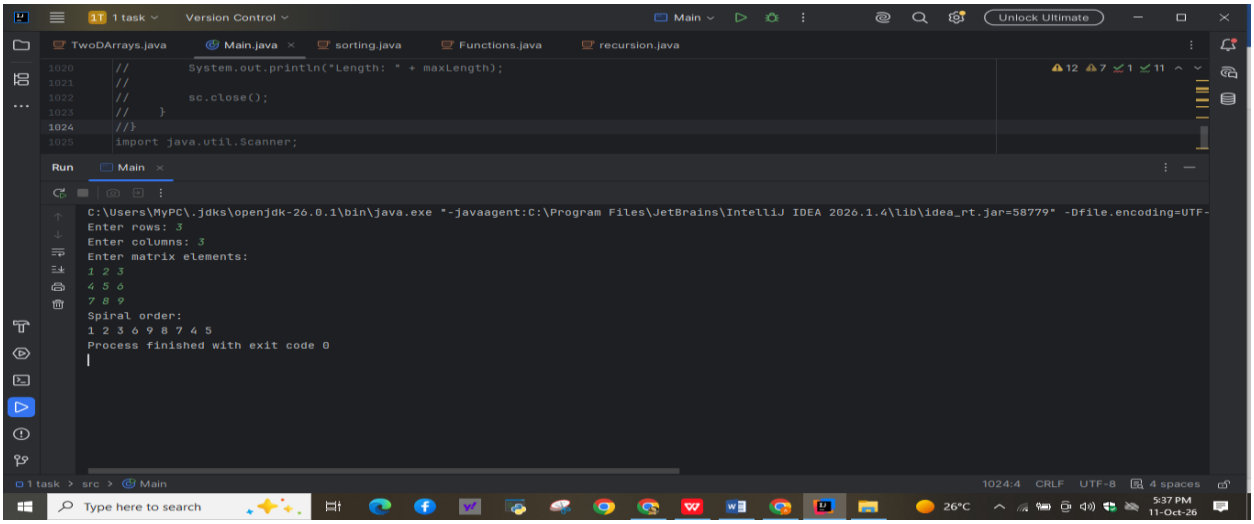
            // 2. Print right column (top to bottom)
            for (int i = top; i <= bottom; i++) {
                System.out.print(arr[i][right] + " ");
            }
            right--;

            // 3. Print bottom row (right to left)
            if (top <= bottom) {
                for (int j = right; j >= left; j--) {
                    System.out.print(arr[bottom][j] + " ");
                }
                bottom--;
            }

            // 4. Print left column (bottom to top)
            if (left <= right) {
                for (int i = bottom; i >= top; i--) {
                    System.out.print(arr[i][left] + " ");
                }
                left++;
            }
        }

        sc.close();
    }
}
```

Output:



Task 26:

Solve Tower of Hanoi for n disks. Print every move and verify that the total equals 2^n - 1.

```
import java.util.Scanner;

public class Main {

    static int moves = 0;

    static void towerOfHanoi(int n, char source,
                             char auxiliary, char destination) {

        if (n == 1) {
            System.out.println("Move disk 1 from "
                    + source + " to " + destination);
            moves++;
            return;
        }

        // Move n-1 disks from source to auxiliary
        towerOfHanoi(n - 1, source, destination, auxiliary);

        // Move the largest disk to destination
        System.out.println("Move disk " + n + " from "
                + source + " to " + destination);
        moves++;

        // Move n-1 disks from auxiliary to destination
        towerOfHanoi(n - 1, auxiliary, source, destination);
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of disks: ");
        int n = sc.nextInt();

        if (n <= 0 || n > 30) {
            System.out.println("Enter a number between 1 and 30.");
        } else {
            towerOfHanoi(n, 'A', 'B', 'C');

            long expected = (1L << n) - 1;

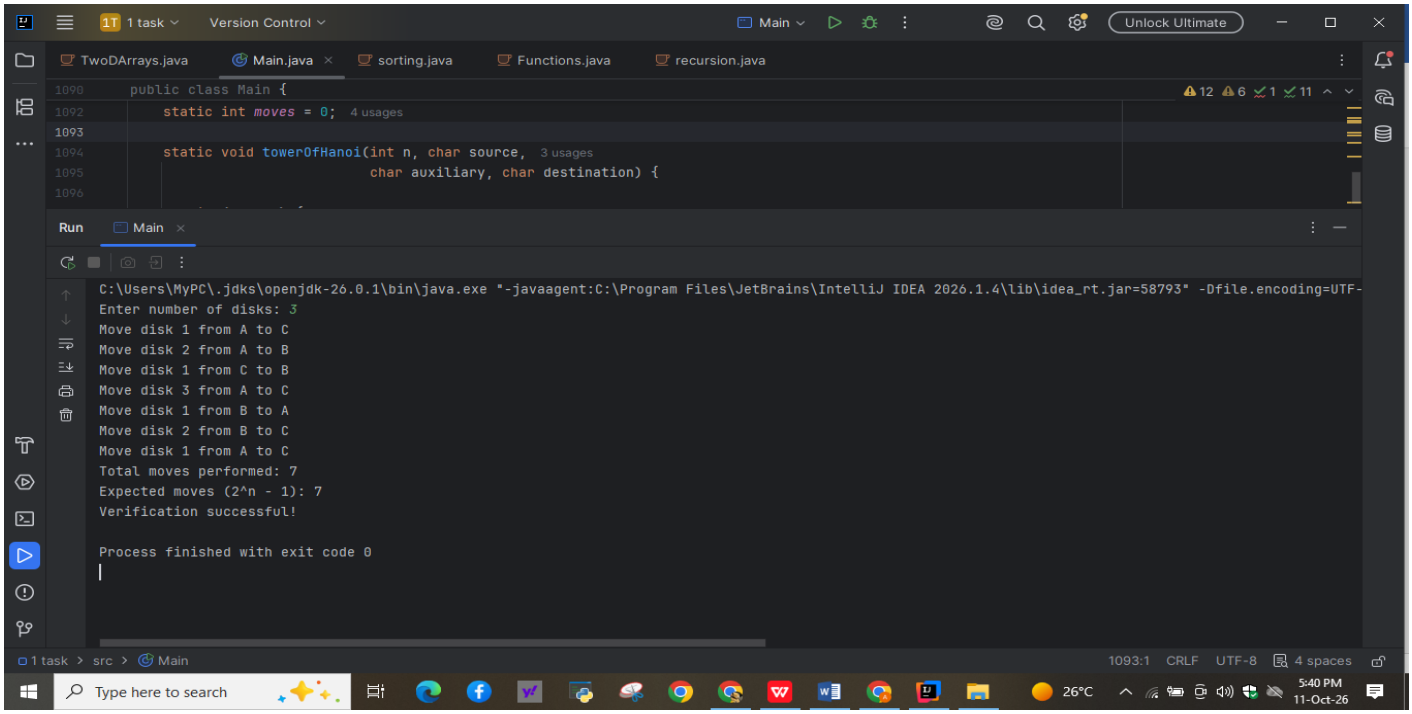
            System.out.println("Total moves performed: " + moves);
            System.out.println("Expected moves (2^n - 1): "
                    + expected);

            if (moves == expected) {
                System.out.println("Verification successful!");
            } else {
                System.out.println("Verification failed!");
            }
```

```
        }

        sc.close();
    }
}
```

Output:



Task 27:

Merge two sorted int arrays into one sorted array in O(n + m) time without calling any sort method.

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter size of first array: ");
        int n = sc.nextInt();

        int[] a = new int[n];

        System.out.println("Enter first sorted array:");
        for (int i = 0; i < n; i++) {
            a[i] = sc.nextInt();
        }

        System.out.print("Enter size of second array: ");
```

```
        int m = sc.nextInt();

        int[] b = new int[m];

        System.out.println("Enter second sorted array:");
        for (int i = 0; i < m; i++) {
            b[i] = sc.nextInt();
        }

        int[] merged = new int[n + m];

        int i = 0, j = 0, k = 0;

        // Compare elements from both arrays
        while (i < n && j < m) {
            if (a[i] <= b[j]) {
                merged[k] = a[i];
                i++;
            } else {
                merged[k] = b[j];
                j++;
            }
            k++;
        }

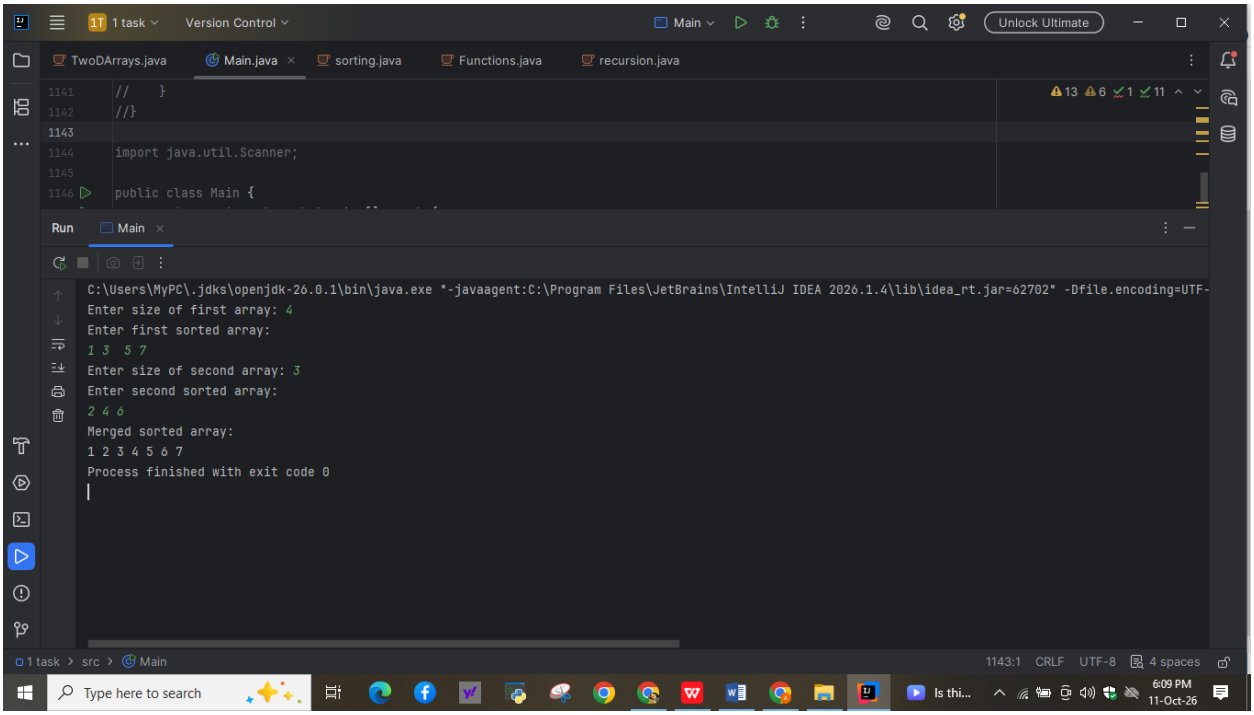
        // Copy remaining elements from first array
        while (i < n) {
            merged[k] = a[i];
            i++;
            k++;
        }

        // Copy remaining elements from second array
        while (j < m) {
            merged[k] = b[j];
            j++;
            k++;
        }

        System.out.println("Merged sorted array:");
        for (int x = 0; x < merged.length; x++) {
            System.out.print(merged[x] + " ");
        }

        sc.close();
    }
}
```

output:



Task 28:

Solve the N-Queens problem for N = 6. Print the total number of solutions and display one valid board.

```
public class Main {

   static final int N = 6;
   static int totalSolutions = 0;
   static int[] queens = new int[N];

   static boolean isSafe(int row, int col) {
      for (int i = 0; i < row; i++) {

         // Same column
         if (queens[i] == col) {
            return false;
         }

         // Same diagonal
         if (Math.abs(queens[i] - col) == Math.abs(i - row)) {
            return false;
         }
      }

      return true;
   }
```

```
    static void solve(int row) {
        if (row == N) {
            totalSolutions++;

            // Save and display the first valid board
            if (totalSolutions == 1) {
                printBoard();
            }

            return;
        }

        for (int col = 0; col < N; col++) {
            if (isSafe(row, col)) {
                queens[row] = col;
                solve(row + 1);
            }
        }
    }

    static void printBoard() {
        System.out.println("One valid board:");

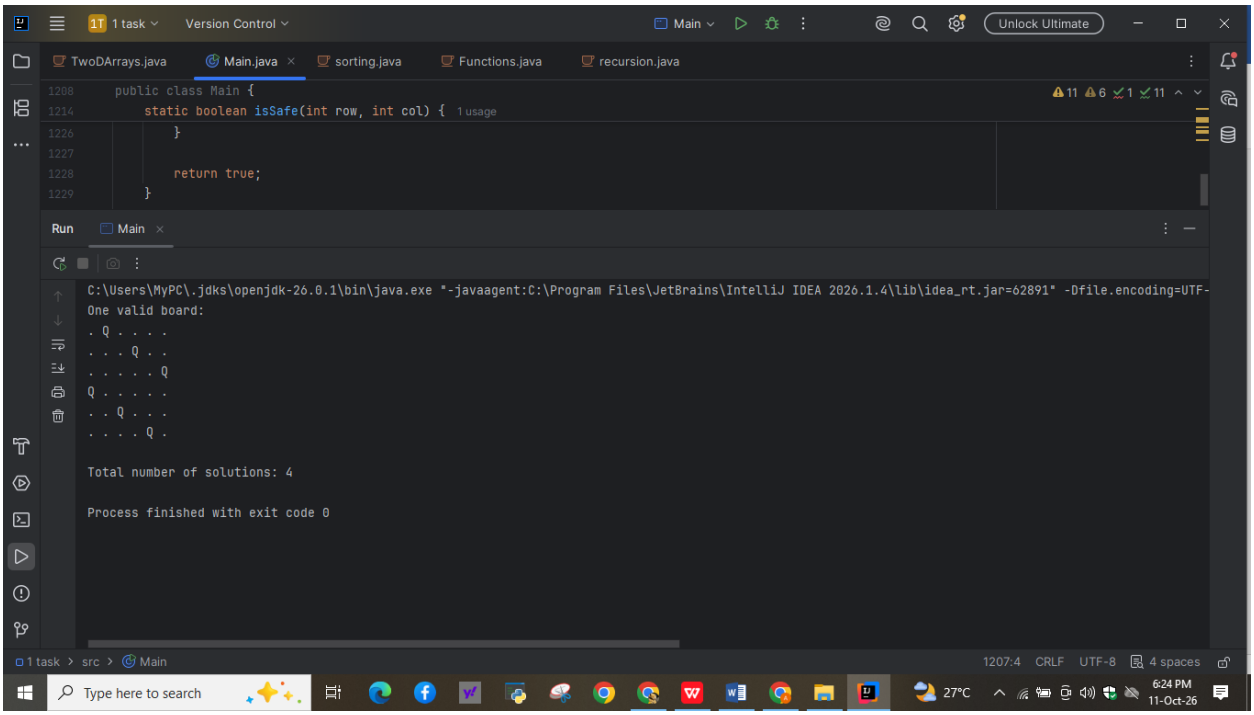
        for (int row = 0; row < N; row++) {
            for (int col = 0; col < N; col++) {
                if (queens[row] == col) {
                    System.out.print("Q ");
                } else {
                    System.out.print(". ");
                }
            }
            System.out.println();
        }

        System.out.println();
    }

    public static void main(String[] args) {
        solve(0);

        System.out.println("Total number of solutions: "
                + totalSolutions);
    }
}
```

Output:



Task 29:

Find the contiguous subarray with the maximum sum (Kadane's algorithm). Print the sum with the start and end indices, and make sure an all-negative array works.

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter array size: ");
        int n = sc.nextInt();

        if (n <= 0) {
            System.out.println("Array size must be positive.");
            sc.close();
            return;
        }

        int[] arr = new int[n];

        System.out.println("Enter array elements:");
```

```
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }

        int currentSum = arr[0];
        int maxSum = arr[0];

        int start = 0;
        int end = 0;
        int tempStart = 0;

        for (int i = 1; i < n; i++) {

            if (arr[i] > currentSum + arr[i]) {
                currentSum = arr[i];
                tempStart = i;
            } else {
                currentSum = currentSum + arr[i];
            }

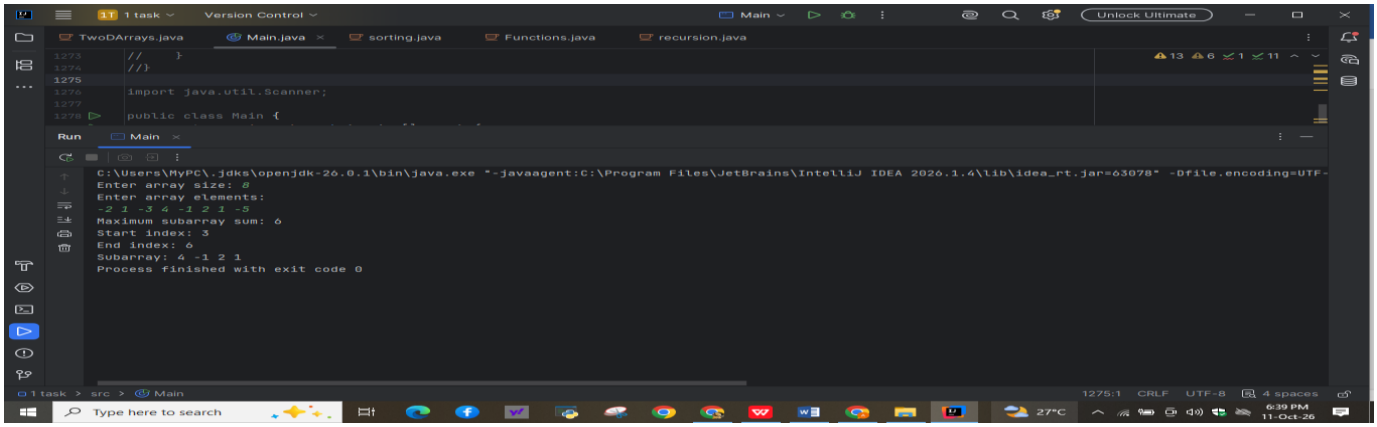
            if (currentSum > maxSum) {
                maxSum = currentSum;
                start = tempStart;
                end = i;
            }
        }

        System.out.println("Maximum subarray sum: " + maxSum);
        System.out.println("Start index: " + start);
        System.out.println("End index: " + end);

        System.out.print("Subarray: ");
        for (int i = start; i <= end; i++) {
            System.out.print(arr[i] + " ");
        }

        sc.close();
    }
}
```

Output:



Task 30:

```
public class Main {

    public static boolean isValidSudoku(int[][] grid) {

        // Check that the grid is 9x9
        if (grid == null || grid.length != 9) {
            return false;
        }

        for (int i = 0; i < 9; i++) {
            if (grid[i] == null || grid[i].length != 9) {
                return false;
            }
        }

        // Check every row and column
        for (int i = 0; i < 9; i++) {
            boolean[] rowSeen = new boolean[10];
            boolean[] colSeen = new boolean[10];

            for (int j = 0; j < 9; j++) {
                int rowValue = grid[i][j];
                int colValue = grid[j][i];

                // Validate row
                if (rowValue < 1 || rowValue > 9 || rowSeen[rowValue]) {
                    return false;
                }
                rowSeen[rowValue] = true;

                // Validate column
                if (colValue < 1 || colValue > 9 || colSeen[colValue]) {
                    return false;
                }
```

```
            colSeen[colValue] = true;
        }
    }

    // Check every 3x3 box
    for (int boxRow = 0; boxRow < 9; boxRow += 3) {
        for (int boxCol = 0; boxCol < 9; boxCol += 3) {

            boolean[] seen = new boolean[10];

            for (int i = boxRow; i < boxRow + 3; i++) {
                for (int j = boxCol; j < boxCol + 3; j++) {

                    int value = grid[i][j];

                    if (seen[value]) {
                        return false;
                    }

                    seen[value] = true;
                }
            }
        }
    }

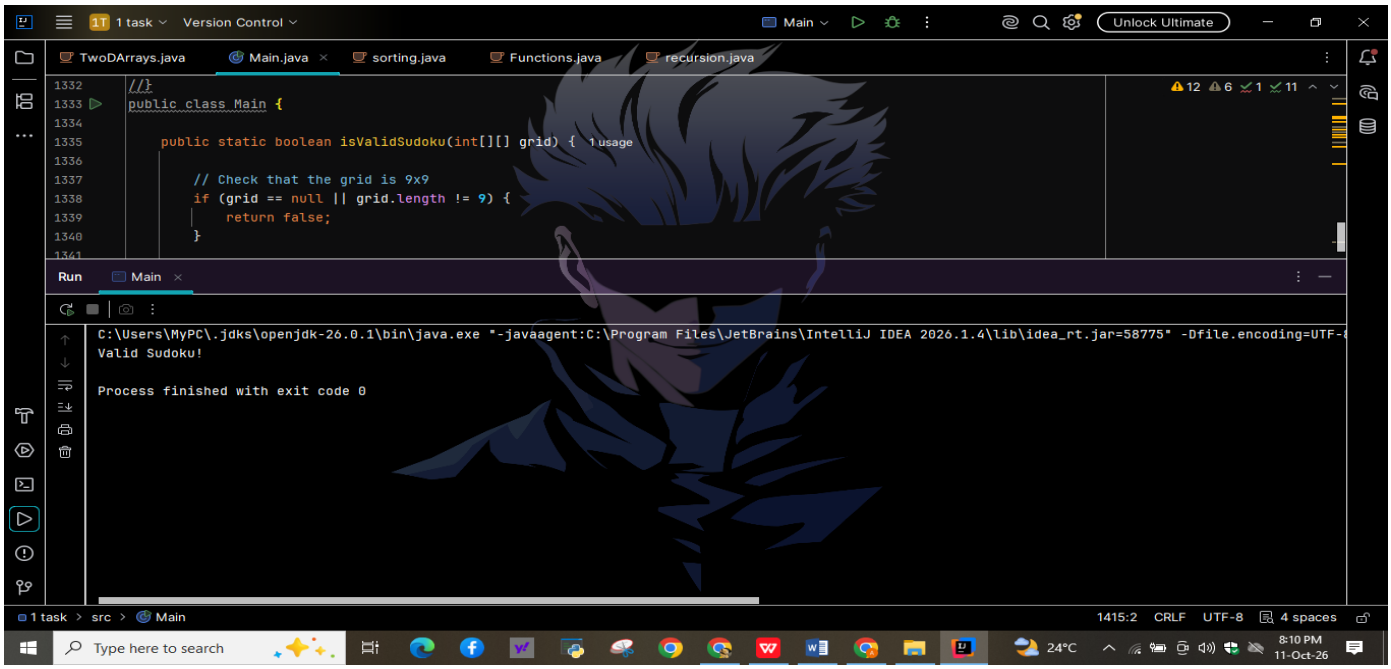
    return true;
}

public static void main(String[] args) {

    int[][] grid = {
        {5, 3, 4, 6, 7, 8, 9, 1, 2},
        {6, 7, 2, 1, 9, 5, 3, 4, 8},
        {1, 9, 8, 3, 4, 2, 5, 6, 7},
        {8, 5, 9, 7, 6, 1, 4, 2, 3},
        {4, 2, 6, 8, 5, 3, 7, 9, 1},
```

{7, 1, 3, 9, 2, 4, 8,

Output:



Task 31:

Compute the factorial of 50 without BigInteger by storing the digits in an int array. Print the full result.

```
public class Main {

    public static void main(String[] args) {

        int n = 50;
        int[] digits = new int[100];

        digits[0] = 1;
        int size = 1;
```

```
        for (int i = 2; i <= n; i++) {

            int carry = 0;

            for (int j = 0; j < size; j++) {

                int product = digits[j] * i + carry;

                digits[j] = product % 10;
                carry = product / 10;
            }

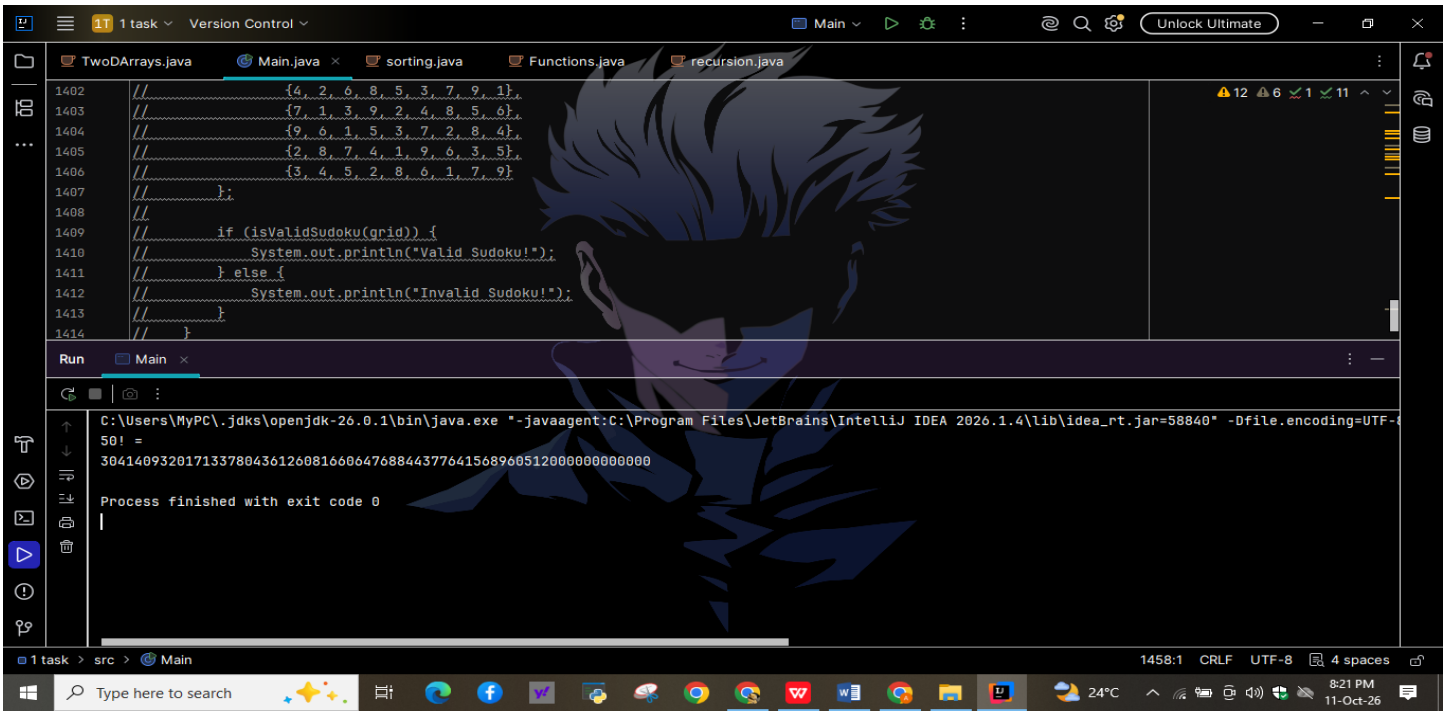
            while (carry > 0) {
                digits[size] = carry % 10;
                carry = carry / 10;
                size++;
            }
        }

        System.out.println("50! = ");

        for (int i = size - 1; i >= 0; i--) {
            System.out.print(digits[i]);
        }

        System.out.println();
    }
}
```

Output:


1 task
Version Control
Main
Unlock Ultimate
TwoDArrays.java
Main.java
sorting.java
Functions.java
recursion.java
//   {4, 2, 6, 8, 5, 3, 7, 9, 1},
//   {7, 1, 3, 9, 2, 4, 8, 5, 6},
//   {9, 6, 1, 5, 3, 7, 2, 8, 4},
//   {2, 8, 7, 4, 1, 9, 6, 3, 5},
//   {3, 4, 5, 2, 8, 6, 1, 7, 9}
//   };
//
//   if (isValidSudoku(grid)) {
//   System.out.println("Valid Sudoku!");
//   } else {
//   System.out.println("Invalid Sudoku!");
//   }
//   }
Run
Main
C:\Users\MyPC\.jdks\openjdk-26.0.1\bin\java.exe "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.1.4\lib\idea_rt.jar=58840" -Dfile.encoding=UTF-
50! =
30414093201713378043612608166064768844377641568960512000000000000
Process finished with exit code 0
1458:1
CRLF
UTF-8
4 spaces
Type here to search
24°C
8:21 PM
11-Oct-26